

Dynamic load balancing for multi-GPU smoothed particle hydrodynamics using two-dimensional staggered domain decomposition

Yelyn Ahn, Namjae Choi¹, Hee Sang Yoo², Eung Soo Kim^{*}

Department of Nuclear Engineering, Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul 08826, South Korea

ARTICLE INFO

Keywords:

Smoothed particle hydrodynamics (SPH)
Dynamic load balancing (DLB)
Multi-GPU parallelization
Staggered domain decomposition
High-performance computing
Large-scale simulation

ABSTRACT

Smoothed Particle Hydrodynamics (SPH) is a Lagrangian particle method widely employed for simulating complex free-surface and multiphase flows. However, its inherently high computational cost and irregular workload distribution make large-scale parallelization particularly challenging. This study presents a novel dynamic load balancing (DLB) algorithm based on a two-dimensional staggered domain decomposition, specifically designed for large-scale SPH simulations on multi-GPU architectures. The proposed method enables two-directional boundary adjustment between GPUs, allowing subdomains to adapt independently along both spatial directions for improved load uniformity. A unified communication routine for multiple neighboring GPUs, a memory-efficient overlapping data structure for particles shared across subdomains, and non-blocking communication to overlap data transfer and computation are also introduced. Further optimizations, including adaptive boundary movement guided by cumulative execution time and dynamic adjustment of neighbor-search cell sizes, enhance scalability and efficiency. Benchmark dam-break simulations demonstrate that the algorithm substantially reduces load imbalance and improves computational performance. Leveraging these advancements, a billion-particle three-dimensional SPH simulation was successfully executed on 56 GPUs, confirming the robustness and scalability of the proposed framework for ultra-large-scale particle-based simulations.

1. Introduction

Smoothed Particle Hydrodynamics (SPH) is a mesh-free, Lagrangian method introduced in 1977 by Gingold and Monaghan [1] and Lucy [2] for astrophysical gas dynamics. In SPH, the fluid is represented by a set of particles that carry field properties (position, velocity, density, etc.) and move with the flow. This meshless formulation naturally handles problems with large deformations, free surfaces, and complex, moving boundaries, making SPH well-suited for multiphase flows [3], violent free-surface flows [4,5], and scenarios with evolving interfaces [6]. A key advantage is that interfaces and free surfaces are captured implicitly by the particle distribution, without the need for explicit interface tracking or re-meshing, which is particularly effective for highly deformable and transient fluid phenomena.

However, these strengths come at a high computational cost, often greater than that of comparable grid-based methods (e.g., Finite Volume or Finite Difference methods). Two primary factors contribute to SPH's

computational intensity [7]: First, each particle interacts with a large number of neighbors (typically 120–400 in three-dimensional simulations), far exceeding the fixed stencil size of grid-based schemes. Second, most SPH formulations in fluid dynamics use a weakly-compressible approach, which requires very small time steps to satisfy the Courant–Friedrichs–Lewy (CFL) condition for stability. The combination of a wide interaction neighborhood and tiny time increments leads to long runtimes, especially as the number of particles increases. Consequently, despite its flexibility, SPH has historically been limited in practice to moderate problem sizes due to these performance constraints.

Over the past two decades, extensive research has focused on accelerating SPH computations, exploiting both software and hardware advances [8–18]. On the algorithmic front, several complementary efficiency-enhancement strategies have been developed to reduce the absolute computational workload. These include coupling SPH with mesh-based solvers (e.g., hybrid SPH–finite difference method (FDM)) to restrict particle-based computations to complex regions [19,20], as

^{*} Corresponding author.

E-mail address: kes7741@snu.ac.kr (E.S. Kim).

¹ Present Address: Idaho National Laboratory, 1955 N. Fremont Ave, Idaho Falls, ID 83402, United States

² Present Address: CEA, DES, IRESNE, Cadarache, Saint-Paul-lès-Durance, France

well as multi-resolution or adaptive-resolution SPH models that dynamically adjust particle sizes to balance accuracy and cost [21,22].

Parallel to these algorithmic improvements, hardware acceleration using Graphics Processing Units (GPUs) has proven particularly effective for particle-based methods. GPUs, equipped with thousands of parallel cores, are particularly well suited for particle-based methods such as SPH. Early work by Amada et al. [23] in 2004 demonstrated one of the first GPU-based SPH implementations, and subsequent studies reported speedups exceeding $100\times$ over CPU-only versions [24]. A GPU's architecture is well-matched to SPH's localized computations, as each particle's interactions can be handled concurrently across many threads. Modern GPUs offer massive parallel throughput and high memory bandwidth, allowing millions of particle updates to be computed in parallel. Despite this, single-GPU simulations face inherent limits: the memory on a single GPU (currently up to $\sim 40\text{--}80$ GB, or about 288 GB for the largest configurations via specialized hardware) bounds the total number of particles that can be stored, and execution time grows rapidly with problem size. This means that even with GPU acceleration, extremely large SPH simulations remain very time-consuming or impossible on one GPU. To push into this regime, multi-GPU (and multi-node) parallelization has become essential.

In a multi-GPU SPH framework, the simulation domain is partitioned into sub-domains, each managed by a different GPU, and a Message Passing Interface (MPI) layer handles data exchange between GPUs. By distributing particles across multiple GPUs, this approach increases the aggregate available memory and computational power, enabling larger simulations and faster wall-clock times. Large-scale SPH studies in recent years have demonstrated the potential of this approach. Valdez-Balderas et al. [25] extended the single-GPU Lagrangian weakly compressible SPH code DualSPHysics to a multi-node, multi-GPU implementation using MPI, while Verma et al. [26] developed a multi-GPU implementation of the Predictive-Corrective Incompressible SPH (PCISPH) method, enabling efficient large-scale incompressible flow simulations. These efforts underscore that multi-GPU SPH can reach unprecedented scales.

However, they also reveal a new challenge. As particles move and the flow evolves, the workload per sub-domain can become severely imbalanced in large-scale SPH simulations. This behavior arises regardless of whether a standard, hybrid, or multi-resolution SPH algorithm is employed, as continuous particle migration and evolving flow structures tend to induce hardware-level load imbalance. As a result, some GPUs may host substantially more particles or incur higher computational cost than others. When one GPU lags behind, all others must idle at synchronization points, causing the overall simulation to progress at the speed of the slowest device. In addition, excessive particle accumulation within a single sub-domain can lead to memory overflow on that GPU, while memory on other devices remains underutilized, effectively limiting the maximum problem size that can be simulated.

To address these issues, researchers have developed dynamic load balancing (DLB) algorithms that adaptively redistribute particles or adjust sub-domain sizes during runtime. The objective is to maintain an even distribution of workload across GPUs (or CPU processes), ensuring synchronized progress among all devices while preventing any single device from being overwhelmed by memory demands. Several approaches have been explored in the literature. Previous research by Oger [27] proposed a CPU-based load distribution algorithm using the Orthogonal Recursive Bisection (ORB) method for simulating fragmented fluid domains like jet breakup. In their study, they conducted simulations utilizing 32,768 cores. The JOSEPHINE [28] code, developed for analyzing unsteady incompressible fluids, incorporates a load balancing algorithm that considers the load differences between neighboring subdomains. The simulation was performed with 5000 particles using four processors. Ferrari et al. [29] performed three-dimensional single-phase simulations using a DLB algorithm based on one-dimensional (1D) domain decomposition. The GPU-based SPH

load balancing algorithms in prior research are as follows. Dominguez [30] presented simulations using 64 GPUs by developing a DLB algorithm that divides the computational domain into one dimension. Verma [31] proposed a DLB algorithm for the PCISPH by dividing the domain into 1D subdomains and conducting a dam-break simulation using 3 GPUs.

While 1D domain decomposition has been widely adopted in SPH parallelization, it suffers from poor scalability as the number of sub-domains increases. When dozens or hundreds of GPUs are used, 1D slicing produces highly elongated subdomains with large surface-area-to-volume ratios. Consequently, particles must communicate across lengthy boundaries, significantly increasing communication overhead and synchronization cost. As a result, the benefits of adding more GPUs are often diminished, motivating the use of higher-dimensional domain decomposition strategies.

Various alternative partitioning strategies have therefore been investigated. Geometry-based approaches such as space-filling curves (SFC) and ORB have been widely adopted in SPH parallelization [27, 32–34]. SFC methods map multi-dimensional space onto a 1D ordering, enabling efficient repartitioning with relatively low overhead, but often suffer from discontinuities and increased communication costs in irregular domains [35,36]. ORB adapts naturally to non-uniform particle distributions by recursively bisecting the domain, yet global rebalancing may require migrating large numbers of particles, and elongated subdomains at high process counts can substantially increase communication overhead [37].

Graph-based approaches follow a different paradigm by representing particles or cells as nodes and their interactions as edges. Partitioning algorithms such as METIS and ParMETIS aim to minimize edge cuts and communication volume, and are effective for irregular workloads. However, constructing and updating interaction graphs is computationally expensive, making frequent repartitioning impractical in large-scale, highly dynamic SPH simulations [38]. As a result, purely graph-based schemes are rarely employed for runtime DLB in SPH.

Conventional two-dimensional (2D) rectangular decomposition employs an aligned grid structure, in which subdomain boundaries are arranged along global row and column directions. Compared to 1D decomposition, this approach reduces the surface-area-to-volume ratio and simplifies parallel implementation through fixed neighbor relationships and static communication patterns. However, this strict boundary alignment also introduces an important limitation for DLB: boundary adjustments are inherently coupled across entire rows or columns. As a result, relieving load imbalance in a single subdomain often induces boundary shifts in other subdomains that do not exhibit load imbalance, leading to unnecessary domain expansion, redundant particle migration, and non-local side effects.

To overcome these limitations, the present study introduces a two-dimensional staggered domain decomposition specifically designed for runtime DLB in large-scale, time-varying multi-GPU SPH simulations. Unlike conventional aligned 2D grids, the staggered layout structurally decouples neighboring subdomain boundaries, allowing boundary adjustments to remain localized. As a result, load imbalance in a single subdomain does not propagate across entire rows or columns. In addition, inter-GPU communication is distributed over multiple non-aligned boundaries, alleviating communication hot spots and improving robustness under highly non-uniform particle distributions.

Based on this staggered domain structure, we develop a runtime DLB framework in which subdomain boundaries are adaptively adjusted to maintain an optimal workload distribution across GPUs. Rather than relying on global repartitioning or large-scale particle migration, the proposed approach achieves fine-grained and localized load redistribution through incremental boundary shifts. This design is particularly well suited for large-scale SPH simulations, where particle distributions evolve continuously due to free-surface motion, strong advection, and mixing. Accordingly, the present work is positioned as a high-performance computing and software optimization study for large-

scale SPH simulations, with a primary focus on multi-GPU parallelization and DLB, rather than on the development of new SPH physical formulations.

Within this framework, the simulation domain is partitioned into a 2D grid of subdomains, and boundary positions are dynamically updated during runtime according to measured GPU execution times at each time step. Several implementation strategies are integrated to enhance scalability and efficiency:

- (i) **Efficient Communication via Single-Array Transfers:** We design a communication scheme where each GPU packs outgoing particle data into a single contiguous array for all its neighbors. This single-array transmission allows one MPI call to handle exchanges with multiple adjacent subdomains, reducing the number of communication routines and minimizing memory overhead. By storing outgoing data for multiple neighbors in an overlapped buffer, memory consumption is further reduced.
- (ii) **Computation–Communication Overlap:** Non-blocking MPI operations are employed to overlap data transfer with computation. While boundary particle data are exchanged asynchronously, GPUs simultaneously compute forces and update particles in interior regions, effectively hiding communication latency and improving overall parallel efficiency.
- (iii) **Execution-Time-Based Cumulative Load Metric:** Previous studies adjusted boundaries through pairwise execution time comparisons between adjacent GPUs, which can accumulate imbalance across many devices. Here, boundary shifts are guided by cumulative execution times relative to the global total, providing globally consistent workload distribution and reducing long-range imbalance.
- (iv) **Dynamic Cell Resizing for Neighbor Search:** An adaptive spatial subdivision is implemented, where the cell size for neighbor searching is dynamically adjusted to optimized values as the simulation evolves. This approach minimizes unnecessary searches and lowers the computational cost per timestep as particle distributions change.

Using these innovations, our 2D load balancing framework continuously redistributes work among GPUs and optimizes performance throughout the simulation. In addition, we adopt a boundary-shifting strategy that transfers particles to neighboring regions based on global load comparisons, which requires less communication than full repartitioning. We validated the algorithm on challenging three-dimensional dam-break simulations with approximately 79 million and 210 million particles, respectively. The dynamic load balancer significantly reduced the load imbalance, as evidenced by more uniform GPU runtimes and the elimination of memory bottlenecks where one GPU would otherwise run out of memory. This led to substantial improvements in computational efficiency and scaling. In addition, to push the limits of the method, we conducted ultra-large-scale tests with one billion particles using 56 GPUs. To the best of our knowledge, this study is the first to achieve a billion-particle SPH simulation using a fully dynamic 2D domain decomposition, where inter-GPU boundaries are adaptively adjusted in both coordinate directions. This milestone establishes a new benchmark in large-scale SPH computation.

The successful execution of a one-billion-particle simulation with our method demonstrates its robustness and scalability under extreme conditions. These findings establish that the proposed 2D DLB approach can effectively harness multi-GPU systems for ultra-large SPH problems, enabling high-fidelity fluid dynamics simulations that were previously infeasible. The algorithm not only improves parallel efficiency but also sets a benchmark for the practicality and extensibility of SPH-based analyses, demonstrating sustained computational efficiency and robust performance in ultra-large-scale simulations.

2. Smoothed particle hydrodynamics

2.1. Basic concept

Smoothed Particle Hydrodynamics (SPH) is a method for obtaining approximate numerical solutions of the fluid dynamics equations by replacing the fluid with a set of particles. These particles act as interpolation points, and each carries individual quantities, such as mass and velocity, following the governing equations. Approximate numerical solutions are obtained based on these particles through kernel approximation and particle approximation. In SPH, the field function f at point x can be expressed using kernel approximation as follows.

$$f_i \approx \int_{\Omega} f(\mathbf{r}) W(\mathbf{r}_i - \mathbf{r}, h) d\mathbf{r} \quad (1)$$

where Ω is the domain of interest at x , $W(\mathbf{r}_i - \mathbf{r}, h)$ is kernel weighting function and h is the characteristic length that defines the domain of influence of the kernel function. The kernel function is selected to be a smooth, isotropic, and symmetric within a compact support domain. The weighting function is positive within the domain of influence and decreases monotonically with distance. In the discretized domain, Eq. (1) can be approximated in a discrete form according to the SPH summation as follows:

$$f_i \approx \sum_j f_j W_{ij} \frac{m_j}{\rho_j} \quad (2)$$

where j is the neighboring particle of particle i , m_j is the mass of the particle j , ρ_j is the density of the particle j , $W_{ij} = W(\mathbf{r}_i - \mathbf{r}_j, h)$ and $\mathbf{r}_j$ is the position vector of particle j . The summation is performed over particles j that lie within the support domain of particle i .

2.2. SPH formulations on governing equations

Fluid flows are governed by the Navier–Stokes equations, including the continuity equation and momentum equation. In the Lagrangian framework, these two equations are expressed as follows.

$$\frac{d\rho}{dt} = -\rho \nabla \cdot \mathbf{u} \quad (3)$$

$$\frac{d\mathbf{u}}{dt} = -\frac{1}{\rho} \nabla P + \nu \nabla^2 \mathbf{u} + \mathbf{g} \quad (4)$$

where ρ is the density of the fluid, $\mathbf{u}$ is the fluid velocity vector, P is fluid pressure, ν is kinematic fluid viscosity, and $\mathbf{g}$ is gravity. Furthermore, in the weakly compressible SPH (WCSPH) formulation, the weak compressibility of the fluid is assumed to follow the artificial isothermal equation of state (EOS) as shown below [39].

$$P = \frac{c_o^2 \rho_0}{\gamma} \left[\left(\frac{\rho}{\rho_0} \right)^\gamma - 1 \right] \quad (5)$$

where ρ_0 is the density reference, c_o is the numerical speed of sound, and γ is an empirically determined constant. The SPH discretized form of the Navier–Stokes equations is as follows:

$$\left(\frac{d\rho}{dt} \right)_i = \rho_i \sum_j \frac{m_j}{\rho_j} (\mathbf{u}_i - \mathbf{u}_j) \cdot \nabla_i W_{ij} \quad (6)$$

$$\left(\frac{d\mathbf{u}}{dt} \right)_i = - \sum_j \left(\frac{P_i + P_j}{\rho_i \rho_j} \right) m_j \nabla_i W_{ij} + \sum_j \frac{2m_j \mu_{ij} \rightarrow \mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{\rho_i \rho_j (|\rightarrow \mathbf{r}_{ij}|^2 + \eta^2)} (\mathbf{u}_i - \mathbf{u}_j) + \mathbf{g} \quad (7)$$

where ρ_i is the density of the particle i , $\mathbf{u}_i$ is the fluid velocity vector of target particle, $\mathbf{u}_j$ is the fluid velocity vector of neighboring particle j and

μ_{ij} is calculated as the harmonic mean of dynamic viscosity of particles i and j , providing a smoother approximation at the interface. For this study, the Wendland C2 kernel function, which exhibits low numerical instability, is employed as the kernel function [40].

$$W(r, h) = \begin{cases} \frac{5}{4(2h)} \left(1 - \frac{r}{2h}\right)^3 \left(1 + 3\frac{r}{2h}\right) & \text{for 1D} \\ \frac{7}{\pi(2h)^2} \left(1 - \frac{r}{2h}\right)^4 \left(1 + 4\frac{r}{2h}\right) & \text{for 2D} \\ \frac{21}{2\pi(2h)^3} \left(1 - \frac{r}{2h}\right)^4 \left(1 + 4\frac{r}{2h}\right) & \text{for 3D} \end{cases} \quad (8)$$

$$\nabla W(r, h) = \begin{cases} \frac{5}{4(2h)^2} \left(-12 \left(1 - \frac{r}{2h} \right)^2 \frac{r}{2h} \right) & \text{for 1D} \\ \frac{7}{\pi(2h)^3} \left(-20 \left(1 - \frac{r}{2h} \right)^3 \frac{r}{2h} \right) & \text{for 2D} \\ \frac{21}{2\pi(2h)^4} \left(-20 \left(1 - \frac{r}{2h} \right)^3 \frac{r}{2h} \right) & \text{for 3D} \end{cases} \quad (9)$$

2.3. Neighboring particle search

In the SPH method, it is necessary to calculate the interactions between particles within the support domain to update the physical properties of the particles. As particle coordinates are updated with each time step, the neighboring particles change, so at each time step, all particles must identify neighboring particles within a distance of κh , representing the effective range of the smoothing function. (In this study, κ was set to 2.) Calculating distances for all particles and storing interacting particles is costly in terms of computation time and memory. Efficient neighbor search is therefore a crucial component of SPH solvers [41]. Therefore, various algorithms, such as linked-list and tree methods, exist for efficient neighbor search. In this study, the modified linked list method proposed by Green [42] was used, which is commonly employed in most GPU-based SPH simulation.

In the modified linked list method, the entire computational domain is divided into a virtual search grid, as shown in Fig. 1. The grid consists of square or cubic cells with a side length of kh . By using cells, particles can search for interacting particles only within their own cell and immediately adjacent cells, rather than searching through all particles,

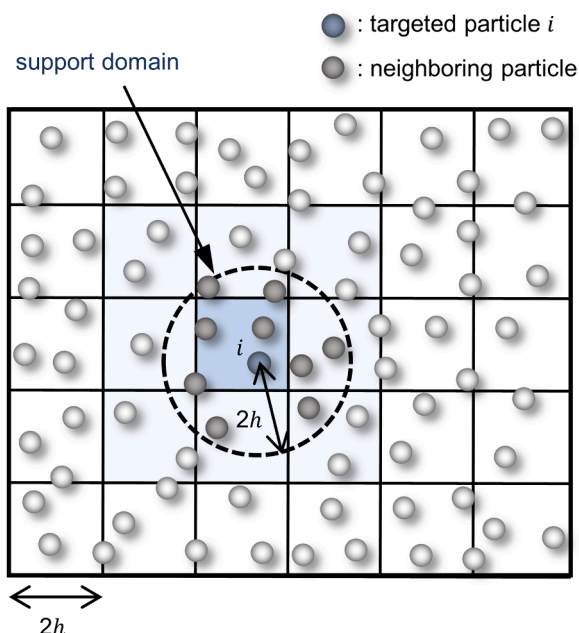


Fig. 1. Cell-based neighbor search in the modified linked list method.

since particles outside their own and adjacent cells are beyond the distance kh . This approach saves significant memory and increases memory access speed by allowing sequential data access.

Each cell in the virtual grid is assigned an index in a predefined order. Next, for each particle, the index of the cell to which the particle belongs is calculated, particles are sorted according to cell indices, and the sorted list is stored in the Particle Index (PID). Variables of particles are sorted in PID order to ensure that all particles within a cell are stored in similar memory locations, as shown in Fig. 2. Additionally, the index of the first and last particle in each cell is stored, eliminating the need to store the index of every particle within each cell. Therefore, when particles search for neighboring particles, they search within their own and immediately adjacent cells based on the generated list to find neighboring particles.

2.4. Time integration

In this study, a fully explicit prediction-correction scheme was utilized for time-stepping. During the prediction step, the particles are advanced by half a time step, and their temporal properties are determined using the following equations.

$$\begin{cases} \mathbf{u}_{t+\Delta t/2} = \mathbf{u}_t + \frac{\Delta t}{2} \left(\frac{d\mathbf{u}}{dt} \right)_t \\ \mathbf{r}_{t+\Delta t/2} = \mathbf{r}_t + \frac{\Delta t}{2} \left(\frac{d\mathbf{r}}{dt} \right)_t \\ \rho_{t+\Delta t/2} = \rho_t + \frac{\Delta t}{2} \left(\frac{d\rho}{dt} \right)_t \end{cases} \quad (10)$$

In the correction step, the acceleration of the particles is determined using their temporal properties, and their velocity and position are subsequently corrected.

$$\begin{cases} \mathbf{u}_{t+\Delta t} = \mathbf{u}_t + \Delta t \left(\frac{d\mathbf{u}}{dt} \right)_{t+\Delta t/2} \\ \mathbf{r}_{t+\Delta t} = \mathbf{r}_t + \Delta t \left(\frac{d\mathbf{r}}{dt} \right)_{t+\Delta t/2} \\ \rho_{t+\Delta t} = \rho_t + \Delta t \left(\frac{d\rho}{dt} \right)_{t+\Delta t/2} \end{cases} \quad (11)$$

2.5. Dam-break test case and pressure analysis

The dam-break problem is widely recognized as a standard benchmark for the quantitative evaluation of numerical methods in fluid dynamics, as it involves strong free-surface impact phenomena and distinct transient pressure responses [43–45]. Although the primary objective of the present study is to assess computational performance, it is first

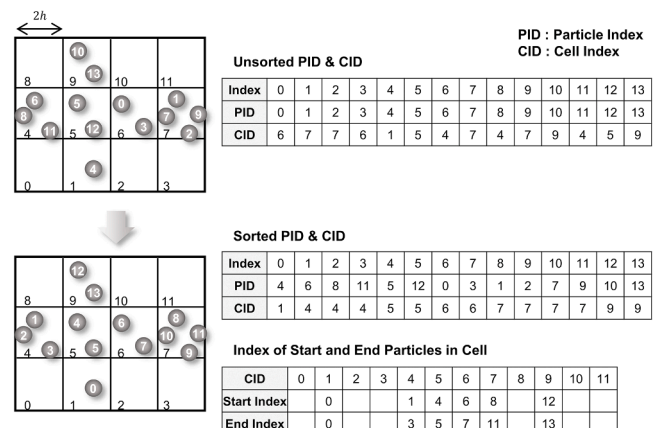


Fig. 2. Illustration of the modified linked list method for neighbor search.

necessary to verify that the numerical model reproduces physically consistent flow behavior. Accordingly, before evaluating computational efficiency, the pressure response is quantitatively examined to establish the numerical reliability of the proposed approach. In particular, the robustness of the WCSPH solver is assessed through an analysis of the temporal evolution of impact-induced pressure.

A three-dimensional dam-break problem interacting with a rigid obstacle is considered. The computational configuration is illustrated in Fig. 3. The densities of both the water and the rigid obstacle are set to 1000 kg/m^3 , respectively, and the dynamic viscosity of water is defined as $0.01 \text{ Pa}\cdot\text{s}$. At the start of the simulation, an initially quiescent water column collapses under gravity and impacts a rigid obstacle located downstream, generating pronounced free-surface deformation and transient pressure loads. To quantitatively capture the pressure evolution during the impact event, pressure measurements are taken on the front face of the obstacle in accordance with the experimental configuration [45]. To evaluate numerical convergence and stability, simulations are performed using three different particle resolutions ($H/\Delta x = 20, 30$, and 40 , where H denotes the tank height and Δx is the initial particle spacing), and the pressure time histories at selected locations (P1 and P5) are analyzed.

Fig. 4 presents a comparison between the numerical pressure time histories and the corresponding experimental data. Fig. 5 shows the pressure field for the finest particle resolution in the dam-break simulation. In the figures, P denotes pressure, g is the gravitational acceleration, H is the initial water column height, ρ is the water density, and t denotes time. The WCSPH simulations successfully reproduce the characteristic pressure response associated with the wave impact, including the rapid pressure rise and subsequent decay. Moreover, as the particle resolution increases, numerical oscillations in the pressure signals are progressively reduced, resulting in smoother pressure profiles and indicating improved stability and convergence of the pressure field computation.

These results demonstrate that the WCSPH formulation employed in this study provides physically consistent and numerically robust pressure predictions for the dam-break problem. This validation establishes a reliable foundation for the subsequent computational performance analysis of the proposed multi-GPU SPH framework.

3. Multi-GPU parallelization of SPH

3.1. Overview

In multi-GPU parallelization, the global computational domain is

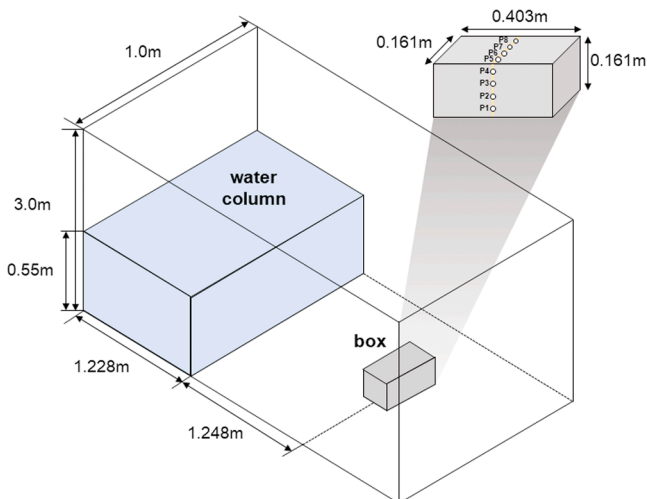


Fig. 3. Computational configuration of the three-dimensional dam-break problem with a rigid obstacle.

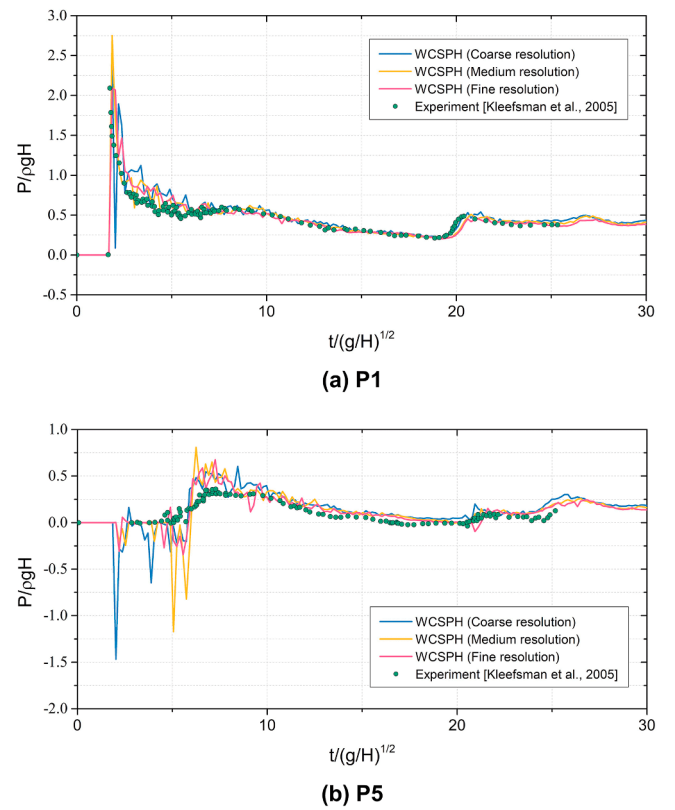


Fig. 4. Pressure time histories at probe locations (a) P1 and (b) P5 for the dam-break problem.

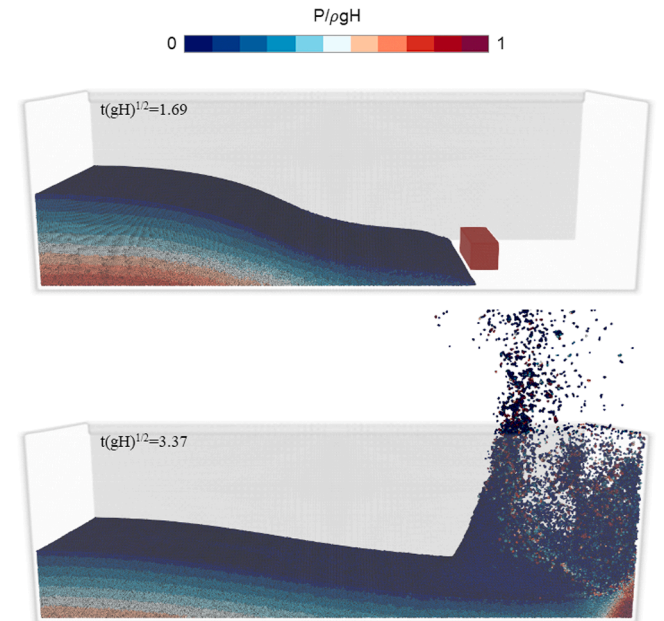


Fig. 5. Pressure field for the finest particle resolution in the dam-break simulation.

divided into subdomains, each assigned to an individual GPU. Communication between GPUs is handled using MPI, which supports both intra-node and inter-node data transfer. This setup not only accelerates simulations but also enables large-scale computations across multiple nodes. Inter-GPU communication is mainly required for two purposes: (i) exchanging neighbor particle data for density and force

calculations near subdomain interfaces, and (ii) migrating particle properties (e.g., position, velocity, density) when particles move across subdomains. These two cases are illustrated in Fig. 6.

To systematically manage communication, each subdomain is divided into three regions, as illustrated in Fig. 7. The edge region (R1) contains particles that must be transmitted to adjacent GPUs, while the halo region (R2) represents the neighboring particles received from other GPUs. The remaining particles form the R0 region, which requires no communication. In this study, it is assumed that particle displacement within a single time step is less than $4h$, a condition guaranteed by the WSPH CFL limit. Based on this assumption, particle data within $4h$ of a GPU boundary are exchanged between GPUs to update the R1 and R2 regions accordingly. For example, GPU 1 transmits its left R1 region to GPU 0 and its right R1 region to GPU 2, while receiving data from GPUs 0 and 2 to form its corresponding R2 regions.

To increase the number of GPUs without sacrificing efficiency, minimizing inter-GPU communication overhead is essential. For this reason, non-blocking communication is employed so that computation and communication can overlap. The overall algorithm workflow is summarized in Fig. 8. At the beginning of each iteration, particle coordinates are updated during the prediction step, and particles are sorted by cell index for efficient neighbor searching (Section 2.3). Density and pressure are then computed for all particles in each subdomain. Force calculation and time integration are performed first for the R1 region, which requires inter-GPU communication. While R1 data are being exchanged, force calculation and integration for the R0 region proceed concurrently. Once both communication and computation are completed, the simulation advances to the next iteration.

3.2. Two-dimensional domain decomposition

3.2.1. Domain decomposition concept

In parallel particle-based simulations such as SPH, domain decomposition is a fundamental strategy for distributing computational work while minimizing inter-process communication. The simulation domain is divided into subdomains, each assigned to a GPU or process, which integrates the equations of motion for particles within that region. To enable interactions across subdomains, halo regions are exchanged between neighbors; the associated overhead scales primarily with the number of halo particles that must be communicated, which in turn depends on the size of subdomain interfaces. Thus, an effective decomposition must simultaneously balance workload and reduce communication cost.

Several partitioning schemes have been developed for SPH and related particle methods. SFC, such as Hilbert or Morton orderings, map the multi-dimensional domain onto a one-dimensional sequence of particles. Particles are sorted along the curve, and contiguous segments are assigned to different processors. The method is relatively

straightforward to implement and can be updated efficiently, since repartitioning reduces to sorting particles along the curve and splitting the list, a procedure with low computational and memory overhead. For this reason, SFC-based schemes have been frequently adopted in parallel SPH implementations on multiple GPUs [32,33]. However, SFC partitions can be limited by the geometry of the domain. For instance, the Hilbert curve requires a power-of-two grid, and discontinuities may arise when it is applied to non-power-of-two grids or unstructured meshes [32]. These discontinuities increase edge cuts and communication overhead, which can degrade parallel performance. In addition, the characteristics of the curve often lead to large edge cuts.

ORB represents another widely used geometric strategy. ORB recursively splits the domain with cutting planes along coordinate axes, such that each region contains an equal share of particles [27,34]. This approach naturally adapts to non-uniform particle distributions, producing rectangular subdomains that are generally better balanced than a uniform grid. ORB is deterministic and can track evolving distributions by adjusting cut positions as the simulation progresses. However, global rebalancing with ORB often involves migrating large numbers of particles, since any movement of a cut plane affects all particles on either side [37]. At large process counts, ORB can also yield elongated subdomains with high surface-to-volume ratios, which substantially increase communication overhead.

Graph partitioning methods take a different approach by representing the simulation as a graph, with particles or cells as nodes and interactions as edges. Partitioning algorithms such as METIS or ParMETIS are then used to divide the graph into parts while minimizing edge cuts. This directly targets the reduction of inter-process communication, and such graph-based decompositions are known to produce high-quality partitions with minimal halo regions. However, graph partitioning is computationally intensive and requires global workload information, which limits scalability. Constructing the interaction graph and computing a new partition can be significantly more expensive than geometric approaches, and the overhead becomes prohibitive if frequent rebalancing is required [38]. For this reason, pure graph-based methods are rarely employed for DLB in SPH.

Beyond these general schemes, simpler approaches such as 1D decomposition have been widely used, particularly in early multi-GPU SPH implementations. In this approach, the domain is divided into strips along a single axis, with each subdomain communicating only with its two immediate neighbors. The main advantage of 1D decomposition is its simplicity: the neighbor topology is straightforward, communication is restricted to local pairs, and boundary adjustments for load balancing affect only the two adjacent processes. This makes the scheme easy to implement and efficient for small numbers of processes. However, as the number of subdomains increases, the strips become increasingly elongated, leading to poor surface-to-volume ratios and enlarged halo regions. As a result, the communication cost rises steeply and scalability deteriorates. By contrast, 2D decomposition partitions the domain along both axes, yielding more compact subdomains and reducing the growth rate of the total communication area with increasing process counts. Consequently, 2D decomposition provides substantially better scalability. Fig. 9 illustrates this trend: while the communication areas of 1D and 2D schemes appear similar for a small number of subdomains, the superiority of 2D decomposition becomes clear as the number of subdomains increases, with the total communication area significantly smaller than in the 1D case.

In this work, we adopt a 2D staggered domain decomposition tailored for multi-GPU SPH. The decomposition is performed at the grid-cell level used for neighbor searching, ensuring that each subdomain contains an integer number of cells and simplifying halo exchange. The staggered design means that cut lines in one direction (e.g. y) can vary between columns in the other direction (x), allowing subdomains to adapt independently to local particle densities. We divide the domain in two stages. First, we split along the x -axis into a predetermined number of vertical columns of subdomains. Then each column is subdivided

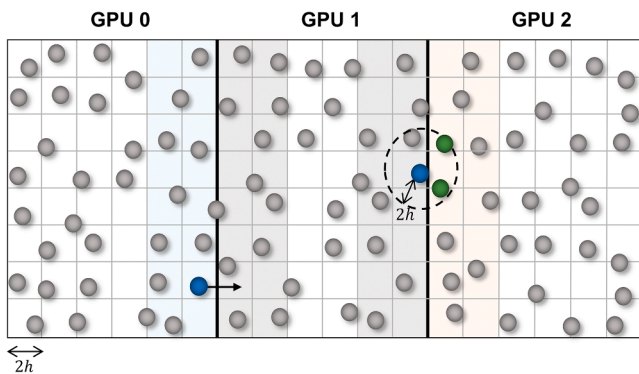


Fig. 6. Inter-GPU communication in multi-GPU SPH simulations. Blue particles denote particles requiring data exchange or migration between GPU subdomains.

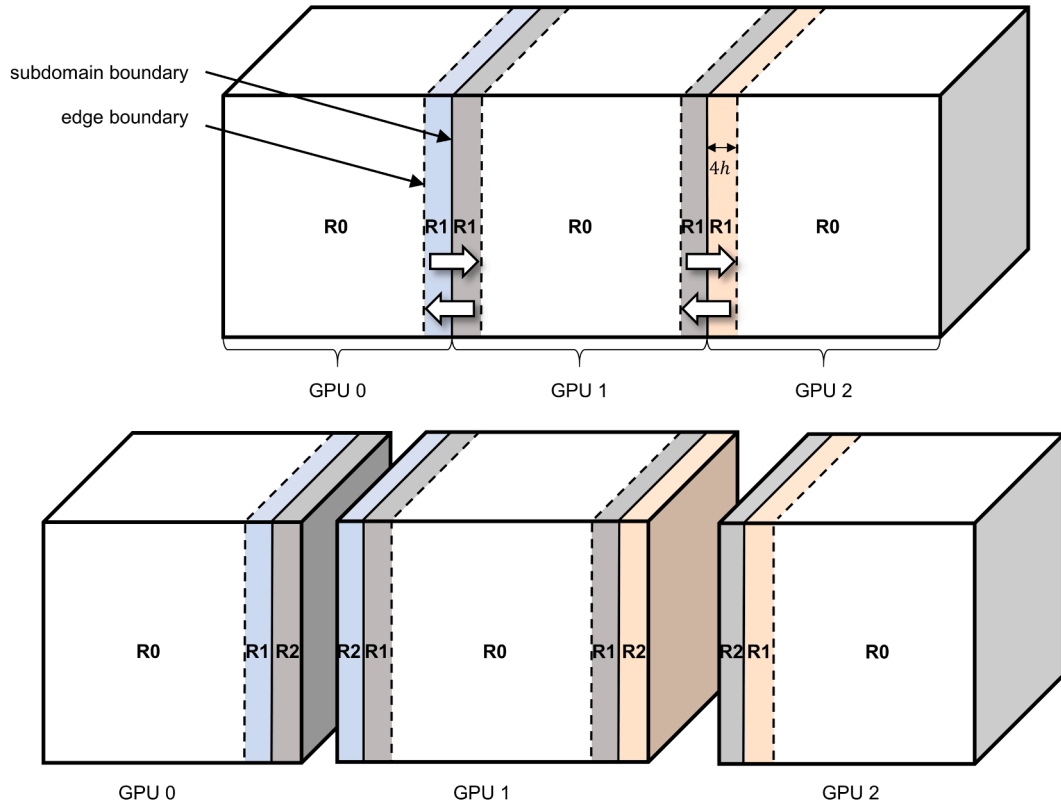


Fig. 7. Partitioning of subdomains into R0, R1, and R2 regions for multi-GPU communication.

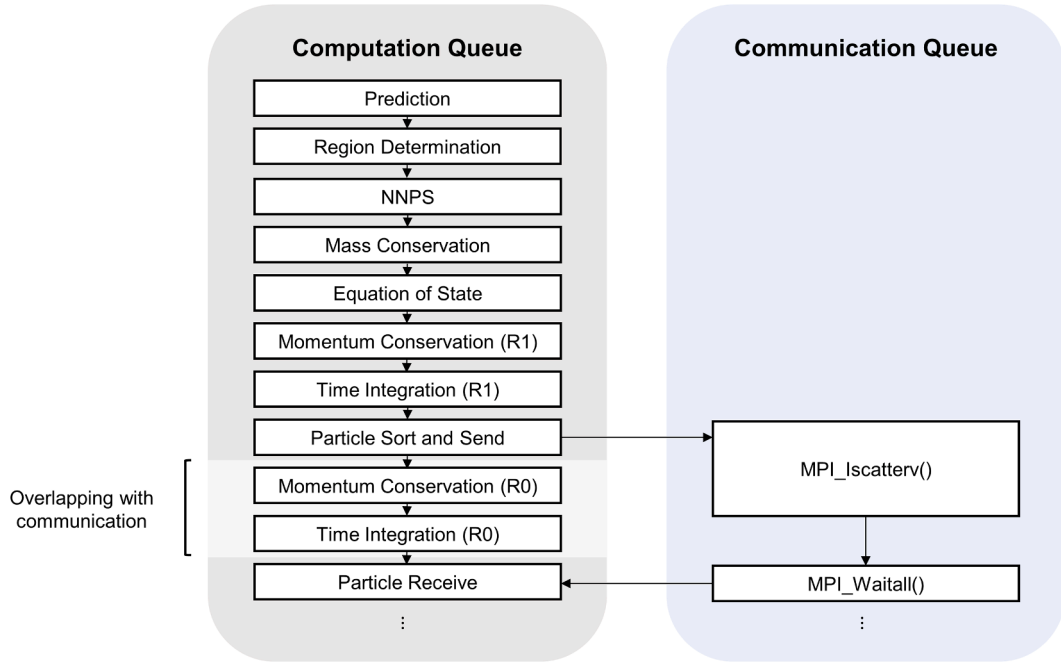


Fig. 8. Workflow of the multi-GPU parallelization algorithm with overlapping computation and communication.

along the y-axis within each column into the required number of rows. For example, Fig. 10 shows a decomposition of the domain into 14 subdomains using 4 columns. Each subdomain also includes halo regions storing neighboring particle data, as illustrated in Fig. 11. Load balancing is performed incrementally by locally shifting boundaries in x or y, requiring communication only between directly affected neighbors. This diffusion-like adjustment avoids the heavy global remapping costs

of methods, while still maintaining compact subdomains with small communication surfaces. Overall, the staggered 2D scheme achieves a practical balance between geometric simplicity, adaptability, and scalability, making it well-suited for ultra-large-scale SPH simulations on multi-GPU architectures.

We initialize the decomposition by distributing particles evenly across GPUs through a two-stage procedure. First, the domain is split

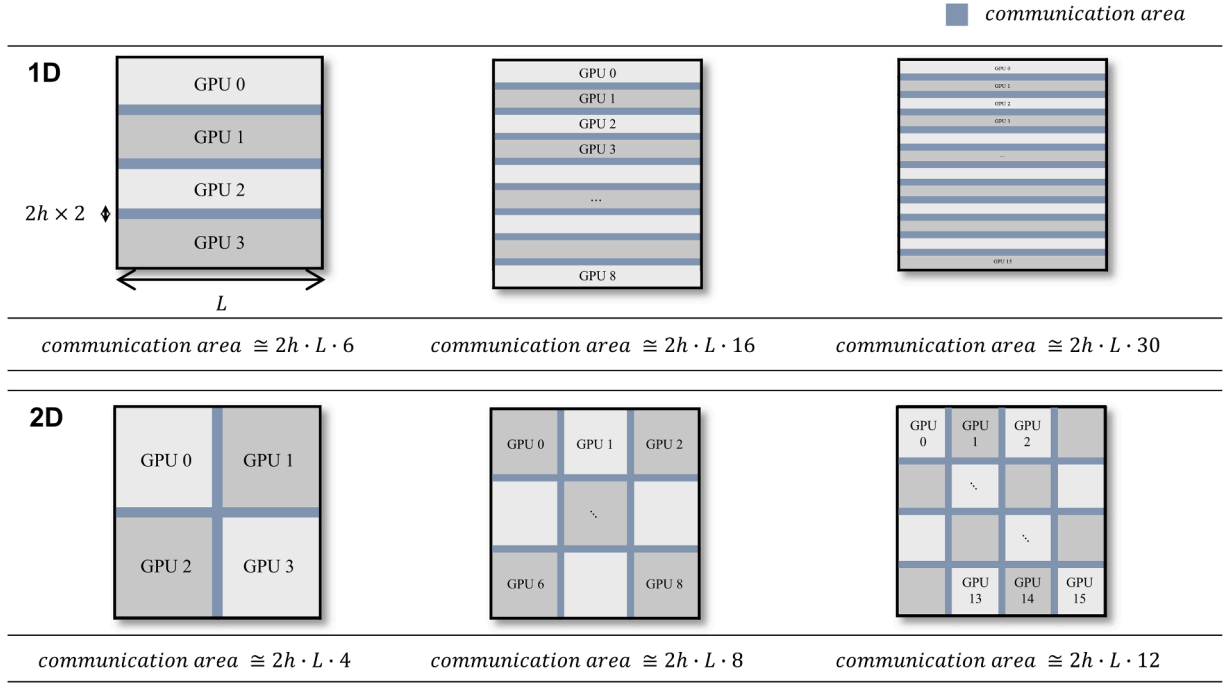


Fig. 9. Comparison of communication areas for 1D and 2D domain decomposition.

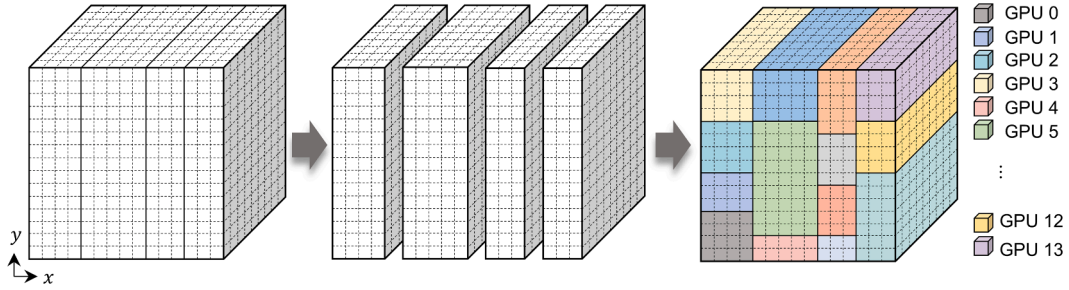


Fig. 10. Example of domain decomposition using 2D staggered domain decomposition method.

into vertical columns, with each column i assigned N_i subdomains so that it contains approximately N_i/N of the total particles. Second, each column is subdivided horizontally so that its N_i subdomains each hold about $1/N_i$ of the column's particles. For instance, in Fig. 10, the vertical boundary between GPU 0 and GPU 4 is positioned such that approximately 4/14 of the particles lie to its left. Subsequent y-direction cuts further equalize the particle counts across GPUs. After partitioning, each GPU's boundary coordinates ($x_{min_boundary}$, $x_{max_boundary}$, $y_{min_boundary}$, and $y_{max_boundary}$) are stored, enabling efficient assignment of particles to their respective regions during input.

3.2.2. Data communication method

In the 2D domain decomposition approach, the communication process is more complex compared to the 1D domain decomposition. In the 1D approach, each particle that requires transmission is sent to only one neighboring GPU. In contrast, in the 2D approach, a single particle may need to be transmitted to up to three GPUs, depending on its location, as illustrated in Fig. 12. In the figure, GPU boundaries are represented by dashed lines. For instance, particles in the blue region of case 1 are transmitted to a single GPU, whereas particles in the green region of case 3 must be transmitted to three GPUs.

Moreover, in the 1D domain decomposition approach, GPU boundaries shift while maintaining sequential connectivity among all regions, which simplifies communication. Particles located at subdomain edges

can always be transmitted to the same neighboring GPU, since the adjacent device number remains unchanged even when the boundaries move. In contrast, in 2D decomposition, boundary shifts modify the set of adjacent devices, thereby complicating the identification of target GPUs for particle transfer. To address this complexity, communication is separated into independent x- and y-direction operations, as illustrated in Fig. 13, and executed through a single function call. In the figure, GPU N is delineated by bold solid lines, while inter-GPU boundaries are represented by bold dashed lines.

For y-direction communication, particle data within a two-cell interval ($4h$) of each GPU's y-direction boundary are transmitted. Specifically, as shown in Fig. 14, particles with y-coordinates less than $y_{min_boundary} + 4h$ are sent to GPU $N-1$, while particles with y-coordinates greater than or equal to $y_{max_boundary} - 4h$ are sent to GPU $N+1$, where GPU N denotes the current domain. In addition, particles that cross the upper boundary are transmitted to GPU $N+1$, while particles that cross the lower boundary are transmitted to the corresponding GPU $N-1$. The indices of particles to be sent are stored in the *send_buffer* array, first for GPU $N-1$ and then for GPU $N+1$.

As for x-direction communication, unlike the y-direction case where adjacent GPU numbers remain continuous, the neighboring GPU indices change during the simulation, requiring explicit identification of adjacent devices. To accomplish this, an array is created to record the GPU numbers to which each cell belongs. For particles in the left R1 region,

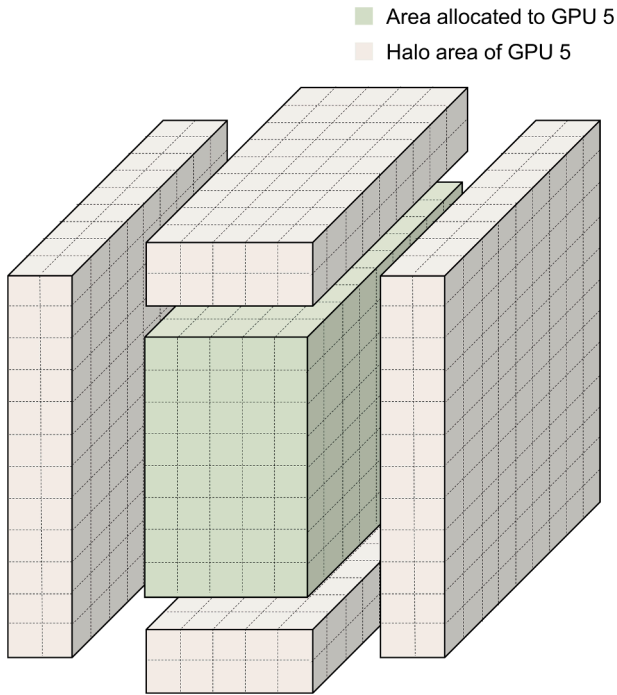


Fig. 11. Allocated and halo regions in the 2D rectangular domain decomposition method.

located within $4h$ of the $x_{min_boundary}$, the destination GPUs are determined by searching the 2×5 cells to the left of their current cell, as shown in the left diagram of Fig. 15. Similarly, for particles in the right R1 region, the destination GPUs are obtained by searching 2×5 cells to the right.

The GPU numbers identified through this search are stored after excluding the GPU number N and its y-direction neighbors GPU $N-1$ and $N+1$. For each particle, at most two GPU numbers can be identified. If two distinct destinations are found, they are summed, and if only a single destination is found, it is doubled. These results are stored in the *gpu_targets* array for each particle. Particles that require transmission are stored with values ranging from 0 to $2 \times (\text{total number of GPU} - 1)$. For particles that do not require transmission, twice the total number of processes is stored to avoid overlap. In addition, particles crossing either boundary, i.e., particles with x-coordinates less than $x_{min_boundary}$ or

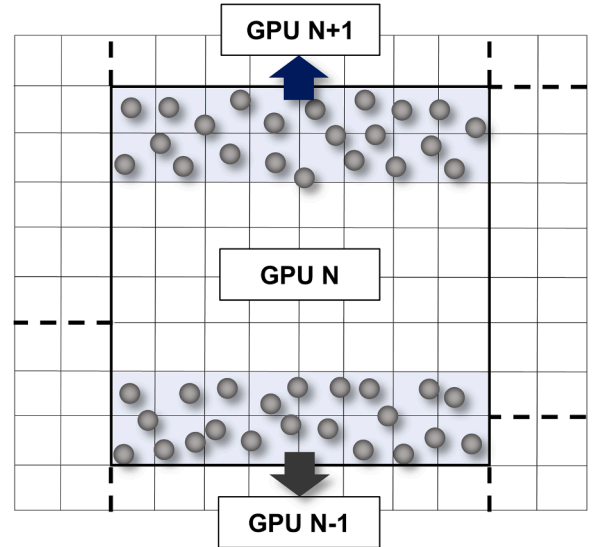


Fig. 14. The scheme of y-direction communication in 2D domain decomposition.

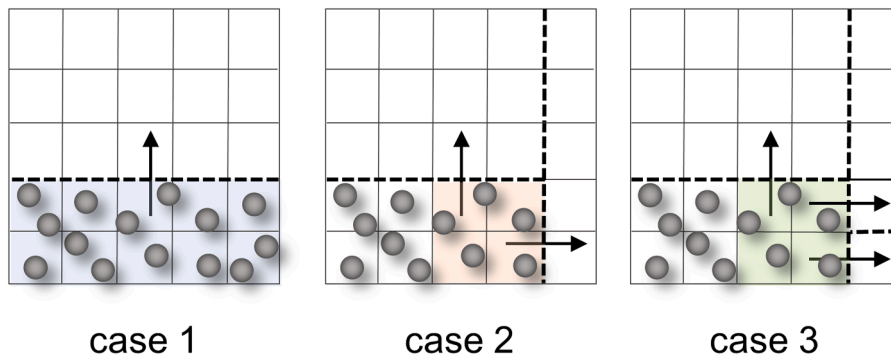


Fig. 12. Particle transmission cases in 2D domain decomposition.

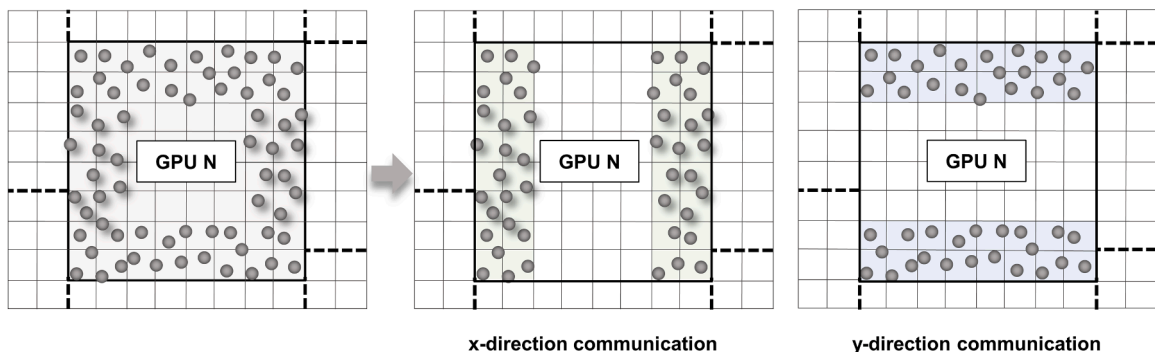


Fig. 13. Communication division in 2D domain decomposition.

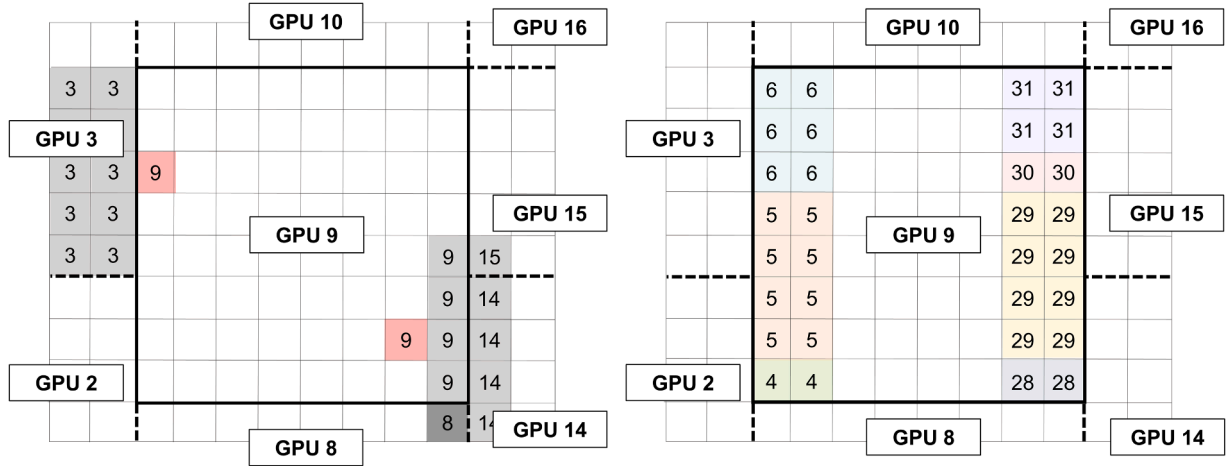


Fig. 15. Dynamic neighboring GPU identification for x-direction communication.

greater than or equal to $x_max_boundary$, follow the same procedure within their respective cells to determine the $gpu_targets$ array as previously described.

Fig. 15 illustrates an example with GPU 9 as the reference domain. In this case, GPUs 2 and 3 are adjacent on the left, while GPUs 14, 15, and 16 are adjacent on the right. In the left diagram, particles in the left red cell lie within the halo region of GPU 3 but not GPU 2. Consequently, the particle data from this cell are transmitted only to GPU 3, and the resulting value stored is 6. In contrast, particles in the right red cell belong to the halo regions of both GPU 14 and GPU 15. Therefore, particle data are transmitted to both GPUs, yielding a combined result stored as the value 29. During the search process, GPU 8 is also detected along with GPUs 14 and 15, but it is excluded since communication with GPU 8 is handled in the y-direction.

As shown in Fig. 16, particle indices are sorted according to the values stored in the $gpu_targets$ array and then appended to the $send_buffer$ after the particle indices already stored for y-direction communication. This sorting ensures that particles destined for the same GPUs are grouped together. For example, particles transmitted to GPU 14 have values 28 and 29 in $gpu_targets$, whereas those for GPU 15 have values

29, 30, and 31, resulting in their grouping within the $send_buffer$ after sorting.

Additionally, by leveraging data overlapping, the size of the $send_buffer$ can be minimized. Particles that must be transmitted to both GPU 14 and GPU 15 are stored only once rather than duplicated. Furthermore, allocating separate memory for each GPU transmission requires additional buffer space for each GPU, which in turn leads to an increase in the overall memory usage. By transmitting particle data to all GPUs from a single memory source, the need for excess allocation is eliminated, and memory usage is minimized.

For communication, the non-blocking collective routine `MPI_Iscatterv` [46] is utilized. This non-blocking function permits computation to proceed concurrently with communication, thereby enhancing computational efficiency. Using the `MPI_Iscatterv` function, the $send_buffer$ is transmitted as a whole, and it necessitates the calculation of the size and displacement of memory to be sent to each GPU. As depicted in Fig. 17, the size and displacement are computed through the send buffer based on the address of the first particle sent to each GPU. Additionally, the memory size sent to GPUs that do not require transmission is set to 0, ensuring efficient execution of the communication process.

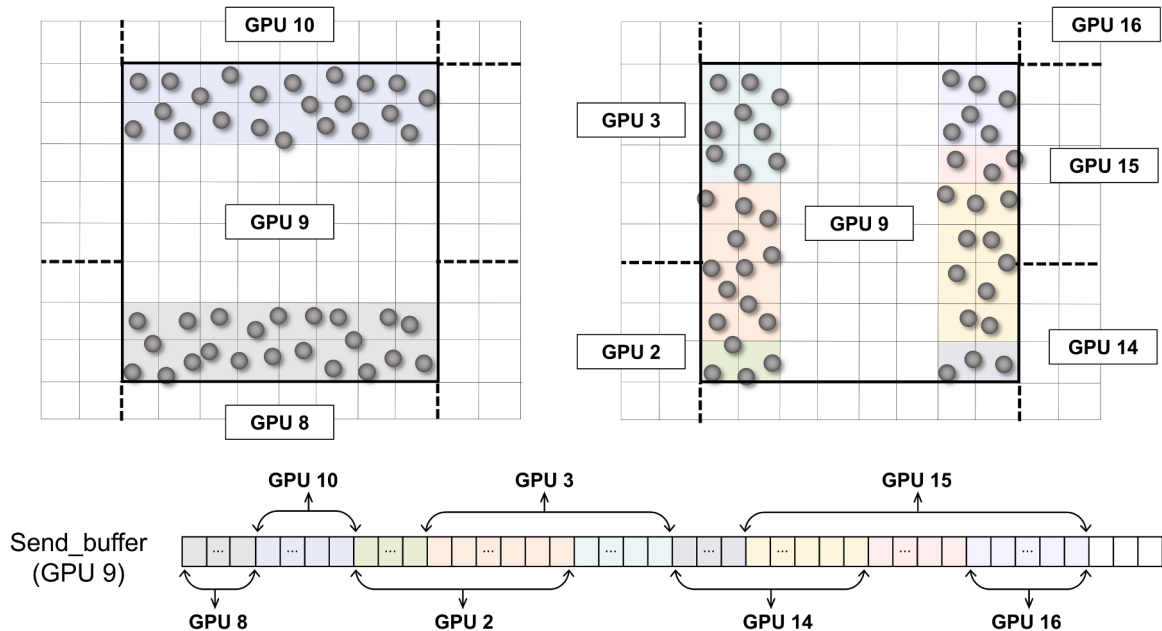
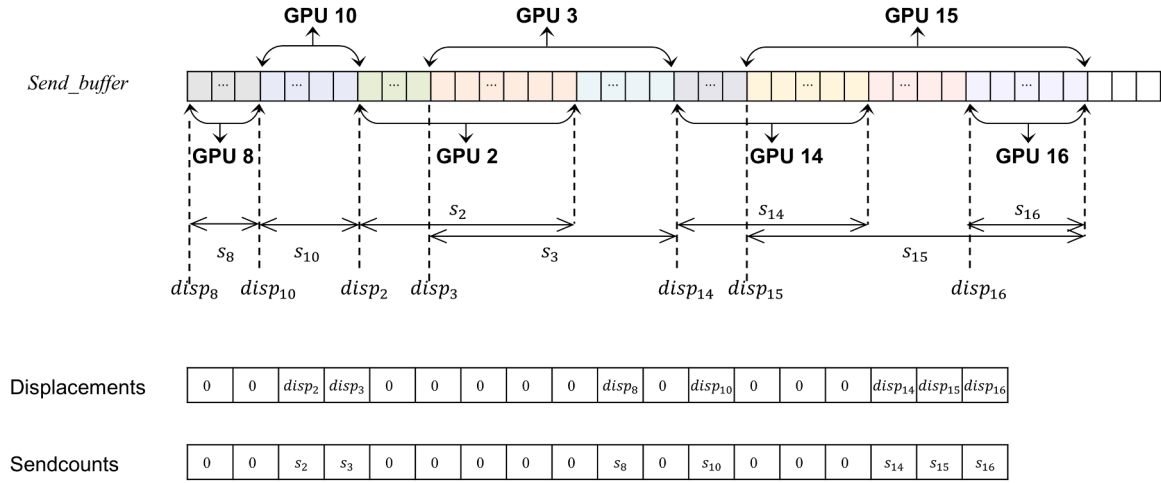


Fig. 16. Memory arrangement for inter-GPU communication.



Displacements : integer array specifying the displacement of each processor

Sendcounts : integer array specifying the number of particles send to each processor

Fig. 17. Calculating displacement and size of memory for transmission to each GPU.

3.3. Optimal cell size control

In the modified linked-list neighbor particle search described in Section 2.3, the computational domain is discretized into a virtual grid composed of cubic cells, and particles search for neighbors only within their own and adjacent cells. The basic cell size is defined as the size of the support domain, $2h$. When the cell size is reduced to $2h/n$ ($n = 2, 3, 4, \dots$), the search area decreases, as illustrated in Fig. 18. However, reducing the cell size also increases the number of cells to be searched, which leads to additional iteration loops and cell management overhead. As a result, an optimal cell size exists that minimizes the total neighbor-search cost by balancing the reduction in search area and the increase in computational overhead. However, in the present study, the optimal cell size is not assumed to exist in a theoretical sense, nor is it derived through analytical modeling. Instead, it is determined dynamically based on direct runtime performance measurements.

Fig. 20 shows the total computation time of a 2D dam-break simulation with 87,789 particles on a single GPU for various values of n . The total computation time is normalized by the case of $n = 1$. The time step is 10^{-5} s, and the initial particle spacing is 0.005 m. The computational geometry is shown in Fig. 19. When the simulation is performed up to 100,000 steps, the total computation time is reduced to approximately 77% at $n = 4$ (cell size = $2h/4$) compared to $n = 1$. However, as the simulation progresses and the particle distribution evolves, the total computation time gradually increases and reaches approximately 87%

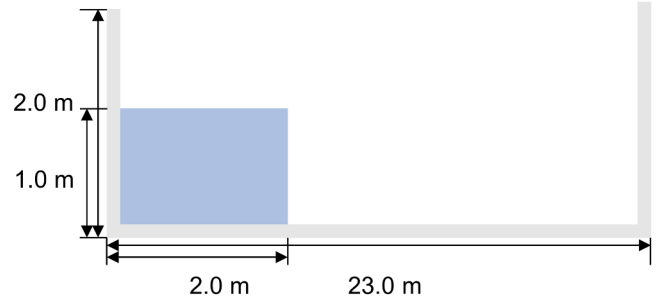


Fig. 19. Computational geometry for 2D dam-break simulation.

at 1,000,000 steps. This behavior indicates that a fixed cell size, even if optimal at an early stage, may become suboptimal as the particle distribution changes.

To address this issue, the optimal cell size is dynamically determined and applied at regular iteration intervals during the simulation. In the present implementation, the cell size is updated as an adaptive procedure executed at fixed iteration intervals (e.g., every 10,000 steps). When the simulation step reaches the update interval, the subsequent time steps are executed sequentially using different candidate values of n ($n = 1, 2, 3, \dots$). Specifically, each candidate n is applied to a single time step, and the execution time of that time step is measured. After

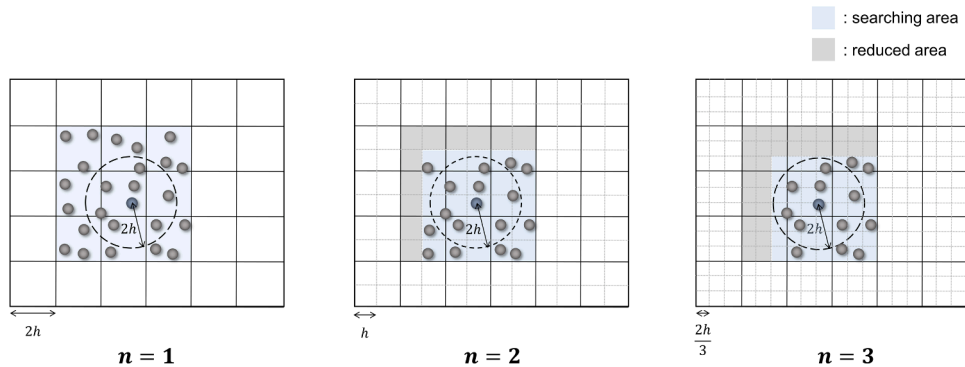


Fig. 18. Search area variation with various cell size.

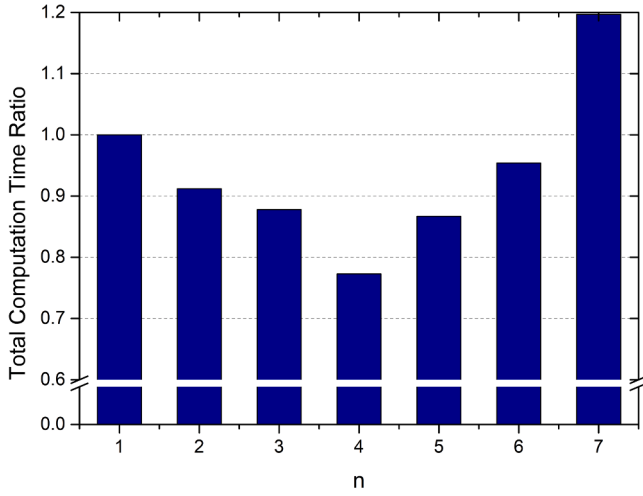


Fig. 20. Total computation time for the 2D dam-break simulation with varying cell size.

evaluating all candidate values, the value of n that yields the shortest execution time for that iteration is selected and used for the subsequent iterations until the next update interval.

Once the optimal n is determined, the cell size is updated to $2h/n$, and the entire Nearest Neighboring Particle Search (NNPS) data structure is rebuilt on each GPU independently. This includes reassigning particles to cells, resorting particles by cell index, and updating the start and end indices of particles in each cell. Since particle-to-cell assignment and linked-list reconstruction are already part of the standard NNPS update, the proposed optimal cell size control does not introduce additional algorithmic overhead, while improving computational efficiency by optimizing the cell resolution. This update is applied independently on each GPU, allowing each device to maintain an optimal balance between search area and cell management overhead according to its local particle distribution.

Fig. 21 compares the total computation time of the fixed $n = 4$ case and the dynamically adjusted cell size case, both normalized by the total computation time at $n = 1$. When the optimal cell size is dynamically selected and applied every 10,000 steps, the total computation time remains nearly constant at approximately 80%. Through this periodic update mechanism, the NNPS structure is continuously adapted to the evolving particle distribution and flow field, preventing performance degradation observed in fixed cell-size configurations and maintaining stable computational efficiency throughout the simulation.

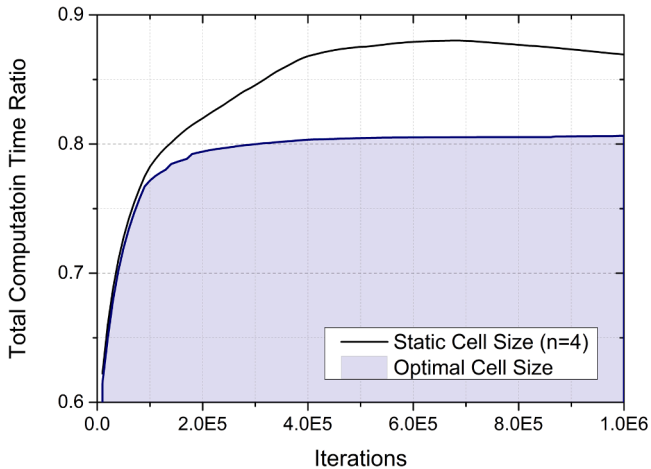


Fig. 21. Comparison of total computation time for static cell size ($n=4$) and optimal cell size over iterations.

4. Two-dimensional dynamic load balancing

4.1. Basic idea

Due to the Lagrangian nature of the method, the SPH code parallelized on multiple GPUs may experience load imbalance as particles traverse GPU boundaries. This can lead to decreased computational efficiency since all GPUs have to wait for the slowest GPU. Additionally, particle count imbalances can result in memory shortages or a decrease in the number of particles available for analysis. To address this issue, we propose a DLB algorithm that redistributes the workload by adjusting subdomain boundaries during simulation to maintain an even distribution among processes. Boundary adjustments are performed by transferring a portion of particles from overloaded regions to neighboring GPUs. Compared to global repartitioning, this approach requires less communication and converges more stably, making it well-suited for problems such as free-surface flows where load distribution changes significantly in both space and time.

Workload representation can be based on either execution time or the number of particles. Using execution time to represent the load offers advantages in two aspects [34]: it allows consideration of varying computational costs for each particle and provides greater flexibility in heterogeneous GPU systems. Therefore, this study considers execution time as the workload representation, focusing solely on the arithmetic part of execution time while excluding communication wait time.

Previous studies [31,47] employed an approach illustrated in Fig. 22, which compares the relative execution times of two adjacent GPUs. In Fig. 22, t_i denotes the execution time of the i -th GPU in a given iteration. This approach is hereafter referred to as the pairwise GPU computation time comparison method. If the relative difference in execution time within the same iteration between the two neighboring GPUs exceeds a prescribed threshold factor ϵ , the boundary is shifted, transferring the R1 region from the GPU with longer execution time to its neighbor with shorter execution time.

However, the pairwise method has a fundamental limitation: it only ensures local balance between adjacent GPUs. As the number of processes increases, small differences can accumulate across the entire system, leading to significant global imbalance. For example, even if all neighboring pairs satisfy the threshold condition, the execution time difference within the same iteration between the first and last GPUs can grow substantially.

To overcome this limitation, in our previous work [48], we proposed the spatially cumulative execution time comparison method. In this method, the cumulative execution time of GPUs to the left of the k -th boundary, $\sum_{i=0}^{k-1} t_i$ is compared with the expected workload share $\frac{k}{n} t_{sum}$, where n is the total number of GPUs and t_{sum} is the sum of the GPU execution times for a given iteration. If the relative difference exceeds a predefined threshold, the boundary is shifted by $4h$ (two-cell intervals), as shown in Fig. 23. When the cumulative execution time is greater than the expected share $\frac{k}{n} t_{sum}$, the edge region of the $(k-1)$ -th GPU is reassigned to the k -th GPU, effectively shifting the k -th boundary leftward by $4h$.

The effectiveness of this method was demonstrated in our earlier study through a 3D dam-break simulation with 21 million particles, in which the computational domain was divided into nine subdomains. While the pairwise comparison maintained local balance, the discrepancy in execution time for a given iteration between the first and last GPUs accumulated to 27.7%, far exceeding the prescribed threshold. By contrast, the cumulative execution time comparison reduced this discrepancy to 6.5%, resulting in a much more uniform execution time distribution across all GPUs.

In the present study, we further extend this approach to a 2D decomposition, in which boundary adjustments are performed along both the x - and y -directions. As shown in Fig. 24, boundaries parallel to the y -axis are adjusted by comparing the sum of execution times of GPUs

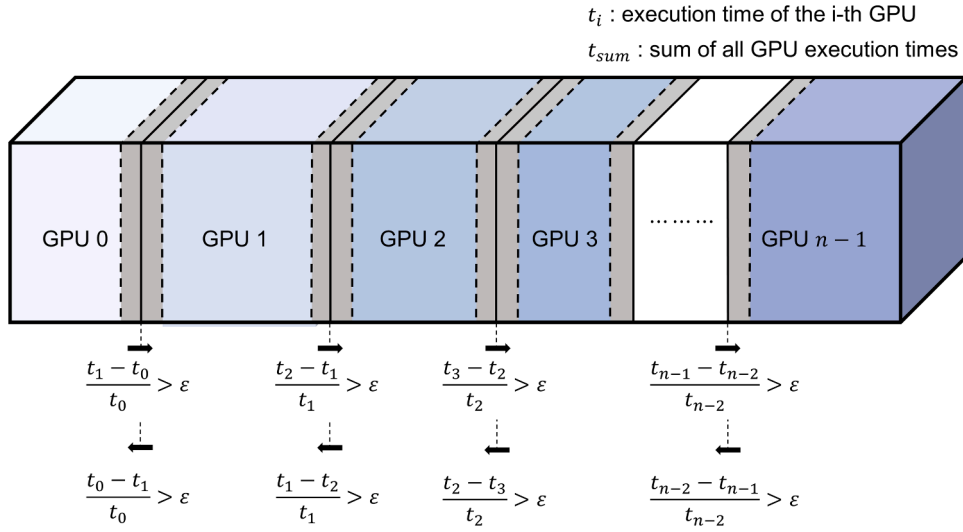


Fig. 22. Schematic diagram of pairwise GPU computation time comparison.

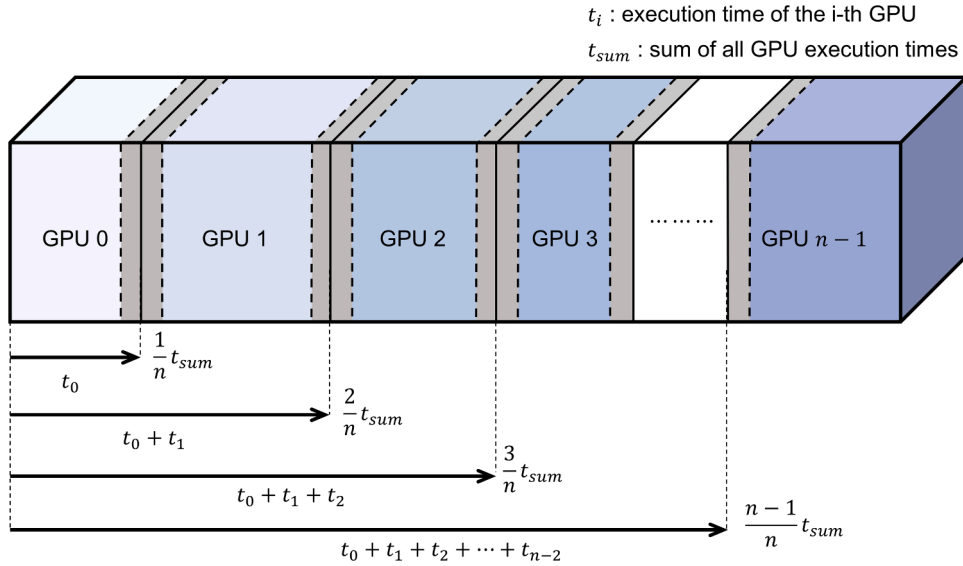


Fig. 23. Schematic diagram of total computation time comparison.

located on the left side of each boundary with the total sum of execution times for the given iteration. Similarly, boundaries parallel to the x-axis are adjusted by comparing the sum of execution times of GPUs located below the boundary within each column with the corresponding sum of GPU execution times for that iteration.

4.2. Implementation of dynamic load balancing

At the beginning of each iteration, the R1 and R2 regions are defined with a 2-cell gap ($4h$), as shown in the left panel of Fig. 25, where each grid cell represents a neighbor-search cell. During the iteration in which GPU boundaries are shifted, the R1 and R2 regions of the subdomains adjacent to the boundary are configured as shown in the right panel of Fig. 25. In this configuration, the expanding subdomain does not allocate an R1 region, while the shrinking subdomain assigns its R1 region to twice the original size.

Fig. 26 illustrates an example where the boundary between GPU 0 and GPU 1 shifts by 2 cells ($4h$), moving from between columns 10 and 11 to between columns 12 and 13. In this case, GPU 0 does not allocate an R1 region, and GPU 1 extends its R1 region to columns 11, 12, 13, and

14. The R2 region for GPU 0 covers columns 11 and 12, while the R2 region for GPU 1 covers columns 9 and 10. After the particle data in the R1 region are exchanged, GPU 0 holds particle data up to column 14, while GPU 1, which does not receive additional data, retains particle information starting from column 11. Consequently, the boundary shifts two cells to the right, now located between columns 12 and 13. This procedure enables boundary shifting to be carried out using the existing communication process, without requiring additional communication overhead.

4.3. Algorithm

Fig. 27 illustrates the overall procedure of the proposed DLB algorithm for each iteration, while Algorithm 1 provides a detailed pseudocode description of the corresponding implementation.

At the beginning of each iteration, particle cell indices are computed from the predicted coordinates, and particles are classified into interior (R0), edge (R1), and halo (R2) regions. Density and pressure are then evaluated for all particles, followed by force computation and time integration for particles in the R1 region. During the time integration

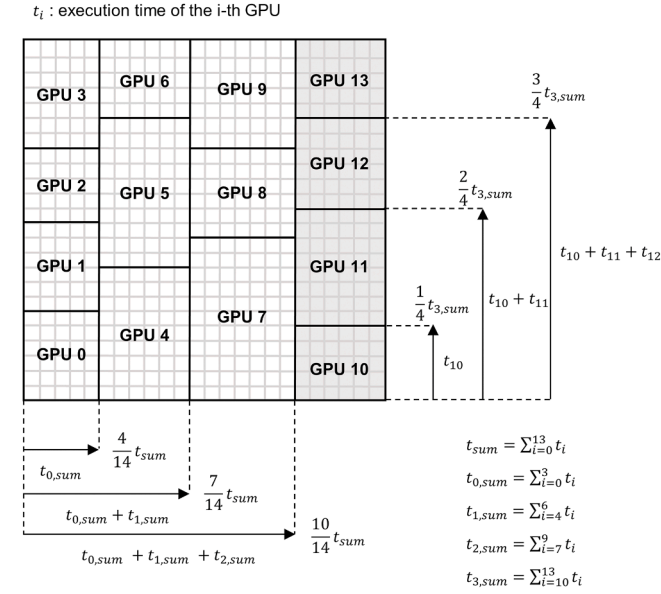


Fig. 24. Schematic diagram of total computation time comparison in 2D domain decomposition.

step, the coordinates and velocities within the R1 region are updated.

After the update, particles requiring inter-GPU communication are identified and sorted, as described in Section 3.2.2. The number of particles to be transmitted and their corresponding displacements are computed. To minimize memory usage and avoid dynamic buffer reallocation, each receiving GPU must know in advance the number of particles it will receive from its neighboring subdomains. This information is exchanged using the collective communication operation MPI_Alltoall [49].

Subsequently, particle data are transferred using non-blocking communication MPI_Iscaatterv, while force computation and time integration for interior particles in the R0 region proceed concurrently, as summarized in Algorithm 1. After the communication is completed, the host process periodically gathers the execution time of each GPU using MPI_Gather [50] to evaluate load imbalance. In the present study, execution times are collected every two iterations.

Based on the execution times measured for the current iteration, the direction of each boundary adjustment is determined. In the subsequent iteration, the R1 and R2 regions are redefined accordingly, and particle exchange is performed using the updated domain boundaries, enabling dynamic boundary movement as illustrated in Fig. 26.

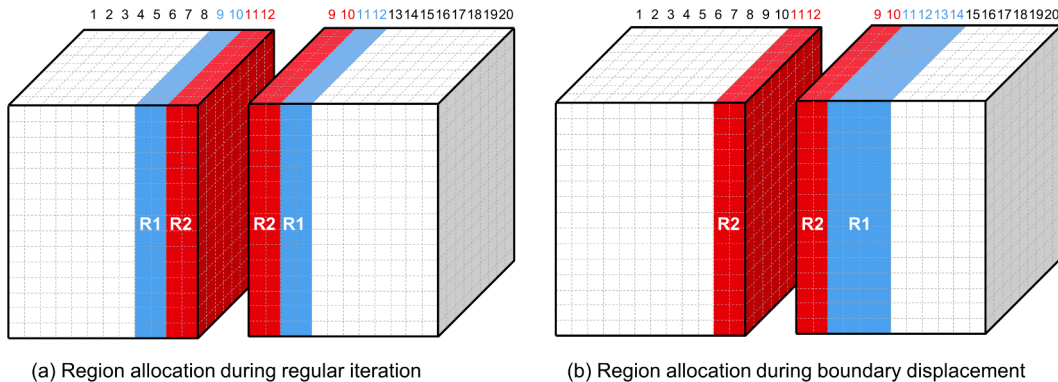


Fig. 25. Configuration of R1 and R2 regions during (a) a regular iteration and (b) an iteration with boundary displacement.

5. Large-scale simulation results with dynamic load balancing

5.1. Simulation configuration

To evaluate the performance of the DLB algorithm using the 2D domain decomposition technique, a 3D dam-break analysis was conducted. The dam-break simulation is particularly suitable for assessing the effectiveness of DLB strategies as it involves significant deformations during execution.

The simulations were conducted on two different GPU systems:

- System A: Equipped with NVIDIA RTX A5000 GPUs, each featuring 8,192 CUDA cores and a memory bandwidth of 768 GB/s. The nodes were interconnected via a 100 Gbps InfiniBand EDR network. The simulation code was compiled using GCC 11.2.0, CUDA 11.6, and OpenMPI 4.1.4 on Ubuntu 20.04.
- System B: Equipped with NVIDIA A100 GPUs, each with 6,912 CUDA cores and a memory bandwidth of 1,555 GB/s. The nodes were connected through a 200 Gbps InfiniBand HDR network. The compilation environment used GCC 4.8.5, CUDA 11.4, and OpenMPI 4.1.1 on CentOS 7.

Fig. 28 illustrates the initial set-up of the simulation. Three different cases were analyzed, and their respective conditions are summarized in Table 1. The total number of particles ranges from 78 million to 1 billion, with the domain size and initial water column dimensions adjusted accordingly.

5.2. Performance evaluation on 25 GPUs

5.2.1. Performance evaluation with 78 million particles

The tank dimensions are 8 m height, 23 m in width, and 23 m in length, while the water has dimensions of 6 m in height, 10 m in width, and 10 m in length. The initial particle spacing was set to 0.02 m, resulting in 78,874,957 particles. The sound speed and time step were chosen as 70 m/s and 10^{-4} s, respectively. For parallelization, the domain was partitioned into five divisions along the x-direction and five along the y-direction, utilizing a total of 25 GPUs. Three simulation scenarios were conducted: (a) without DLB, (b) with 2D DLB, and (c) with 1D DLB. In case (a), the computational domain was divided into equal-sized subdomains, while in cases (b) and (c), a threshold factor of 0.05 was applied.

Fig. 29 presents snapshots of the simulation for case (b), with the left panels showing the domain partitioning. Fig. 30 illustrates the domain partitioning in the 2D DLB simulation, viewed from above, at intervals of 5,000 steps from the initial state to 55,000 steps. To distinguish the partitions, particles are colored according to their processor assignments. As the simulation progresses, the partition boundaries adapt to changes in particle distribution.

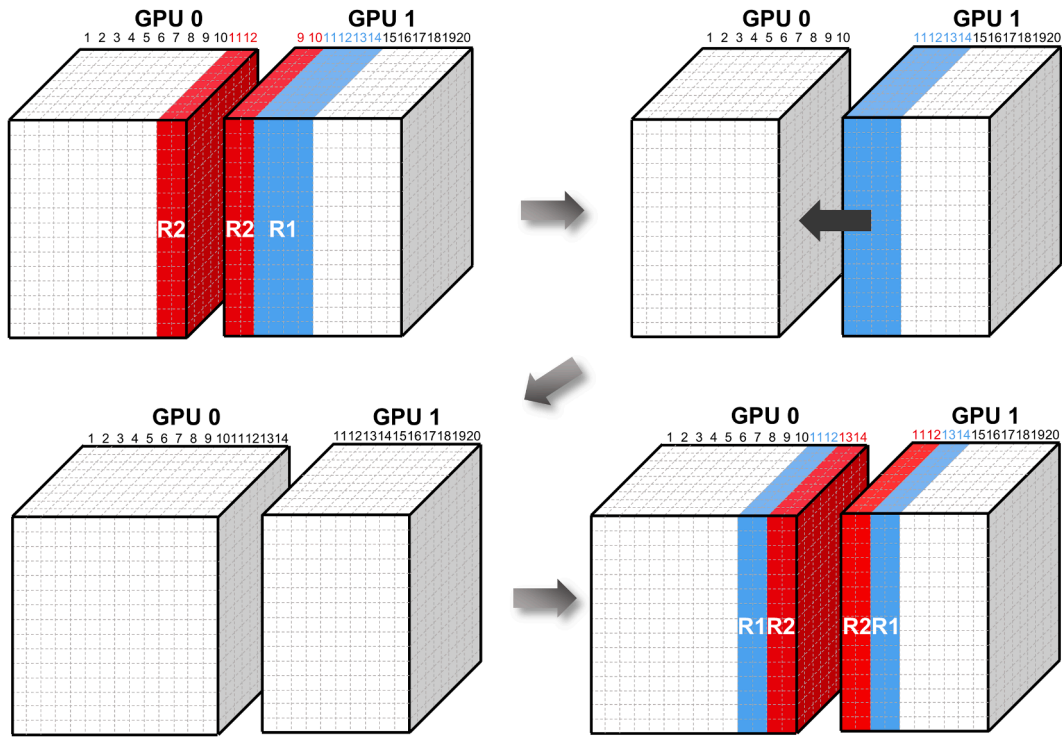


Fig. 26. Boundary movement between GPUs through R1 and R2 region assignment.

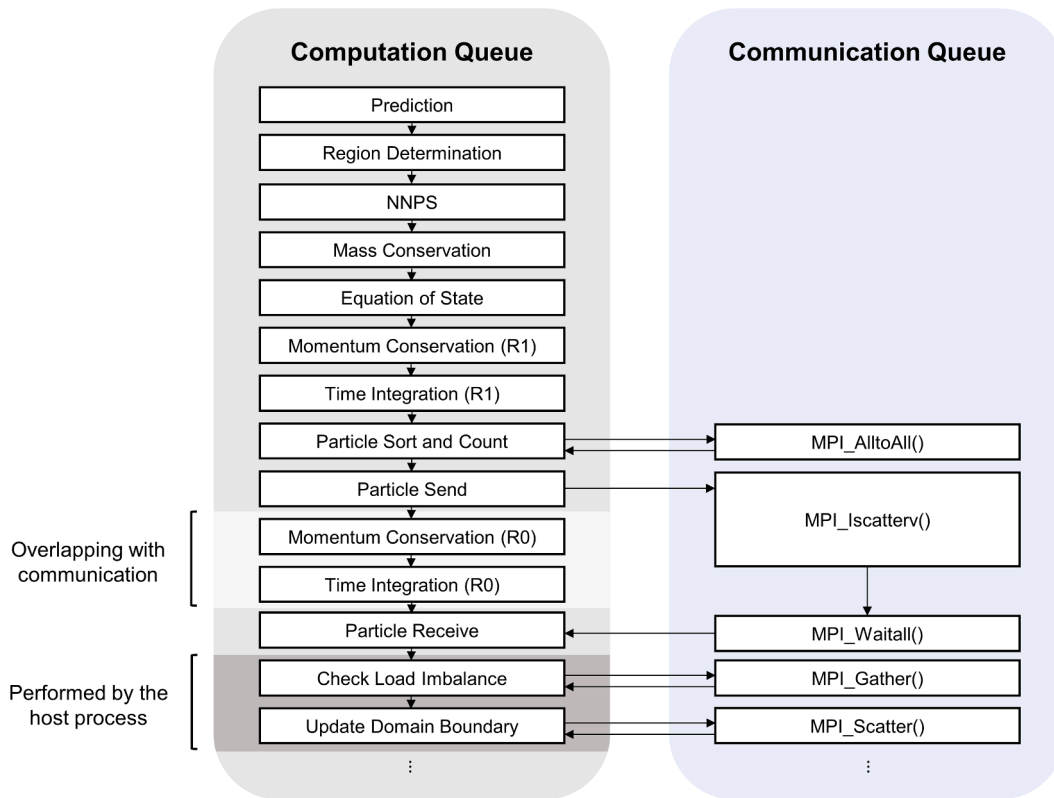


Fig. 27. Flowchart of the SPH DLB framework.

Fig. 31 plots the load imbalance over iterations with and without DLB, and Fig. 32 presents the total computation time, excluding file I/O, plotted as a function of iteration. Unless otherwise stated, total computation time refers to wall-clock time excluding file I/O. The load

imbalance quantifies the disparity in workloads among processes and is defined as the ratio of the maximum execution time for a given iteration to the average execution time for the same iteration:

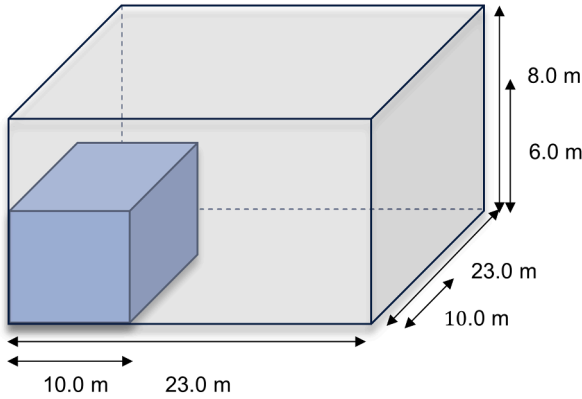
Algorithm 1

Multi-GPU SPH time-stepping with staggered 2D DLB.

```

for  $n = 1, \dots, N_{steps}$  do
  Prediction and region classification (GPU)
    Predict particle positions and velocities
    Classify particles into interior ( $R_0$ ), edge ( $R_1$ ), and halo ( $R_2$ ) regions
  Neighbor search and particle reordering (GPU)
    Assign particles to  $\kappa h$ -sized cells
    Sort particles by cell index (PID) and reorder particle data
    Build cell start/end lists for NNPS
  Density and pressure update (GPU)
    Update density via mass conservation
    Compute pressure using the equation of state (EOS)
  Boundary particle interaction (GPU)
    Compute momentum interactions for edge particles ( $R_1$ )
    Perform time integration for  $R_1$ 
  Boundary data packing (GPU)
    Identify particles to be exchanged with neighboring GPUs
    Pack outgoing particle data into a single contiguous send buffer
  Asynchronous communication (Host / MPI)
    Initiate non-blocking collective communication
    (MPI_Iscatterv)
  Overlapped interior computation (GPU)
    Compute momentum interactions for interior particles ( $R_0$ )
    Perform time integration for  $R_0$ 
  Communication completion (Host / MPI)
    Wait for communication completion (MPI_Waitall)
    Unpack received halo particles
  Runtime load monitoring and balancing (Host)
    Measure per-GPU execution time
    Update staggered 2D domain boundaries if load imbalance exceeds
    threshold
  Maintenance (GPU)
    Update dynamic cell size for NNPS if enabled
end for

```

**Fig. 28.** Initial configuration of 3D dam-break simulation geometry.

$$\text{Load imbalance} = \frac{t_{\max}}{t_{\text{avg}}} \quad (12)$$

where $t_{\max}$ denotes the maximum execution time measured among all processes for a given iteration and t_{avg} represents the average execution time over all processes for the same iteration. A higher load imbalance value indicates a larger workload disparity between processes, while a

value closer to 1 indicates a more balanced workload.

The load imbalance decreased markedly from 1.575 to 1.026 during the simulation, corresponding to a 2.29-fold reduction in the total computation time. In particular, the total computation time up to the highly imbalanced 20,000th timestep was reduced by a factor of 3.57. Before applying the DLB algorithm, as shown in Fig. 31, a pronounced workload concentration on specific GPUs led to substantial imbalance and a sharp increase in total computation time during the early stage of the simulation. As particles dispersed, the imbalance gradually diminished, and total computation time increased more moderately. In contrast, when DLB is applied, the uniformity of execution times measured for each iteration across GPUs is significantly improved throughout the simulation, resulting in a consistently balanced workload distribution. As shown in Fig. 32, the total computation time exhibits a linear increase during the simulation. Fig. 33 presents heatmaps of GPU execution time across iterations as the simulation progresses, highlighting the significant improvement in execution time uniformity achieved through the application of DLB.

Figs. 34 and 35 present the load imbalance and total computation times for the 1D and 2D DLB approaches. The 2D domain decomposition method demonstrates 1.54-fold higher computational efficiency compared with the 1D method, with load imbalance reduced from 1.082 to 1.026. While both methods alleviate imbalance relative to the case without load balancing (1.575), the elongated aspect ratio of the 1D

Table 1

Summary of dam-break simulation conditions for performance evaluation.

Case	Number of Particles	Number of Particles per GPU	GPU Count	System	Domain Size [m]	Water Size [m]	Particle Spacing [m]	Time Step [sec]
1	78,874,957	3,154,998	25	A	23*23*8	10*10*6	0.02	10^{-4}
2	212,570,643	8,502,826	25	A	30*30*2	11*11*1.5	0.01	10^{-4}
3	1,057,443,043	18,882,911	56	B	30*30*7	15*15*4.5	0.01	10^{-4}

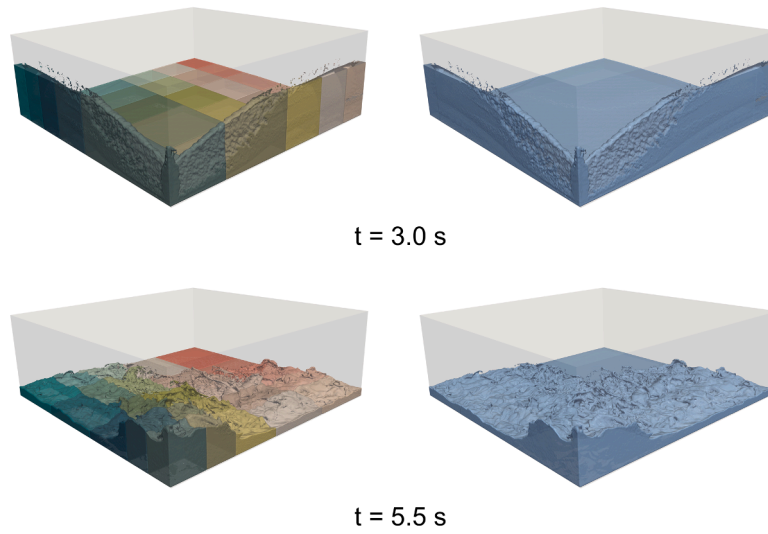


Fig. 29. Snapshots of 3D dam-break simulation using 2D DLB.

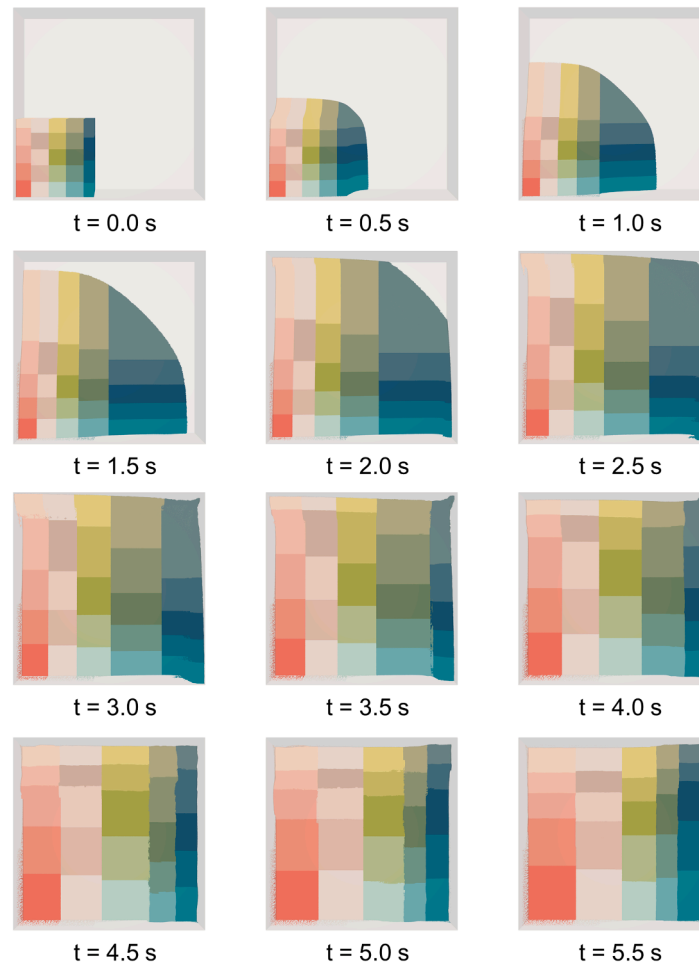


Fig. 30. Snapshots of dynamic load balancing in region partitioning: Top view.

decomposition results in substantially higher communication costs and reduced efficiency.

Quantitatively, the average number of communicated particles per GPU up to 10,000 steps is 159,483 in the 1D approach, compared with 64,963 in the 2D approach, representing a 2.46-fold difference. The larger aspect ratio in the 1D decomposition also increases the number of

halo particles and memory demand. Consequently, when employing a large number of GPUs, the 2D domain decomposition method becomes essential for achieving improved computational performance.

With the 2D DLB algorithm, load balance is maintained consistently throughout the computation, regardless of particle distribution, leading to a uniformly and moderately increasing computation time. The 1D

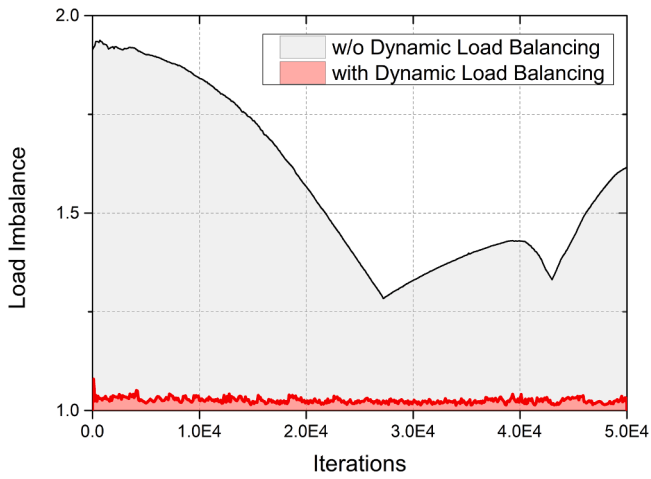


Fig. 31. Load imbalance over iterations with and without 2D DLB.

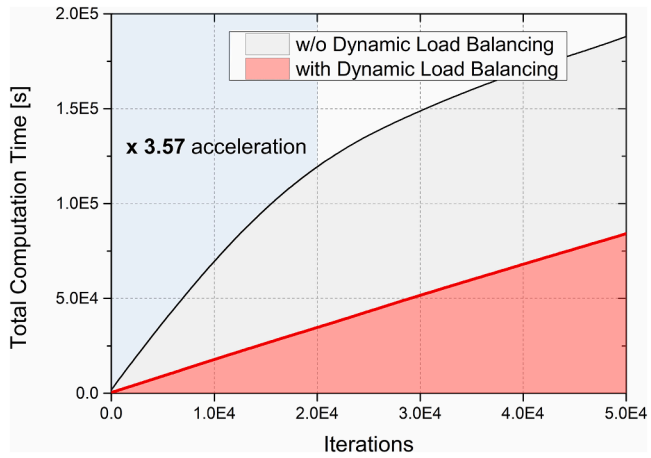


Fig. 32. Total computation time over iterations with and without 2D DLB.

algorithm also reduces imbalance from 1.575 to 1.082 and produces more stable computation times. Nevertheless, the unfavorable aspect ratio in the 1D approach results in considerably higher communication overhead and much lower computational efficiency, as illustrated in Fig. 35.

5.2.2. Performance evaluation with 210 million particles

In the 210 million particle simulation, 25 GPUs were employed, with the computational domain decomposed into five columns along the x-direction and five rows along the y-direction, resulting in a total of 25

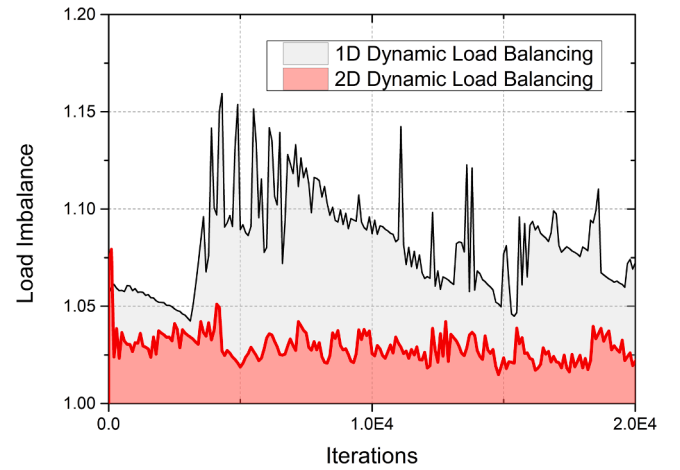


Fig. 34. Load imbalance over iterations for 1D and 2D DLB.

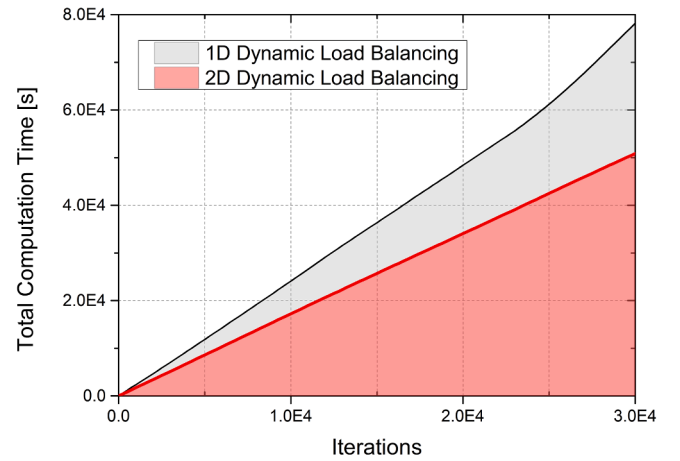
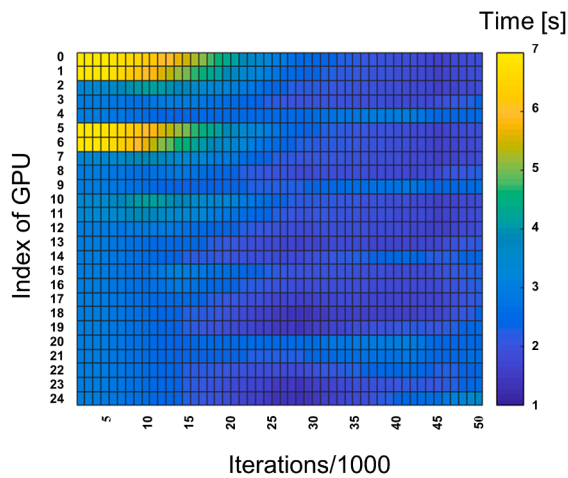
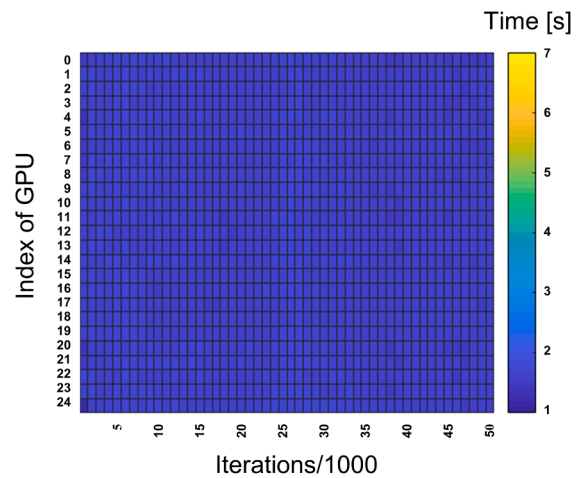


Fig. 35. Total computation times over iterations for 1D and 2D DLB.



(a) without dynamic load balancing



(b) with 2D dynamic load balancing

Fig. 33. GPU execution time heatmaps for each iteration before and after applying DLB.

subdomains. Two simulation scenarios were conducted: (a) without DLB, (b) with 2D DLB.

Fig. 36 shows the minimum–maximum range and mean GPU execution time measured for each iteration, before and after applying the DLB algorithm. The shaded regions represent the range between the minimum and maximum execution times across all GPUs for a given iteration, while the solid lines indicate the average GPU execution time for that iteration. The gray shaded region corresponds to the execution time variation before applying DLB, and the blue shaded region corresponds to the variation after applying DLB. It can be observed that the execution-time range across GPUs, measured for each iteration, is significantly reduced after DLB is applied, indicating improved workload uniformity. Moreover, the average execution time measured for each iteration also decreases, demonstrating enhanced computational efficiency.

Fig. 37 shows the total computation time as a function of iteration. While the total computation time increases linearly in both cases, the slope for the DLB applied case is considerably smaller. This indicates that the total computation time is reduced by a factor of approximately 5.67 times when DLB is applied, highlighting its effectiveness in accelerating large-scale simulations.

Furthermore, Fig. 38 presents the evolution of load imbalance over the simulation time. Without DLB, the imbalance remains persistently high (2.121), whereas the application of DLB maintains the imbalance close to the ideal value of 1.0 (1.026) throughout the simulation. This demonstrates that the proposed load balancing strategy effectively equalizes the computational load among GPUs.

5.3. Ultra-scale simulations on 56 GPUs

To examine the scalability of the proposed 2D DLB algorithm, large-scale simulations with approximately one billion particles were performed using 56 GPUs on System B as described in Section 5.1. The one-billion-particle simulation was conducted to test the limits of multi-GPU SPH parallelization performance and to evaluate the algorithm's scalability and stability under extreme computational conditions. SPH fluid simulations with more than one billion particles remain exceedingly rare due to constraints in computational resources and code scalability. In particular, performing such large-scale simulations, where particle distributions change significantly during the calculation, requires an efficient load balancing strategy that can adapt to drastic variations in workload.

The tank and fluid domains were specified in a rectangular configuration, as shown in Fig. 28, with an initial particle spacing of 0.01 m,

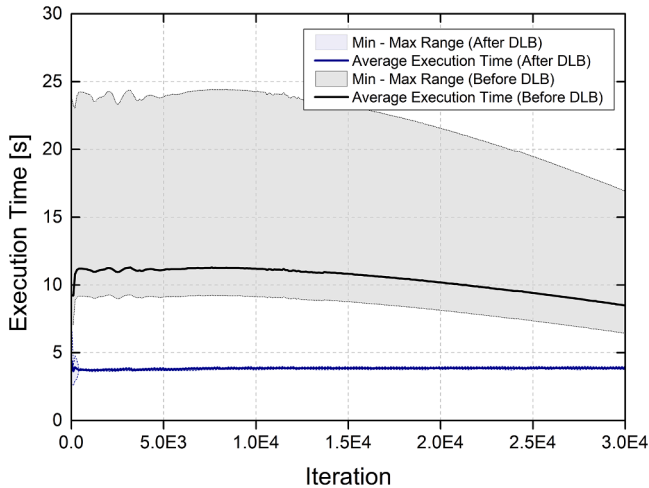


Fig. 36. Comparison of minimum, maximum, and average GPU execution times measured for each iteration before and after applying the DLB algorithm.

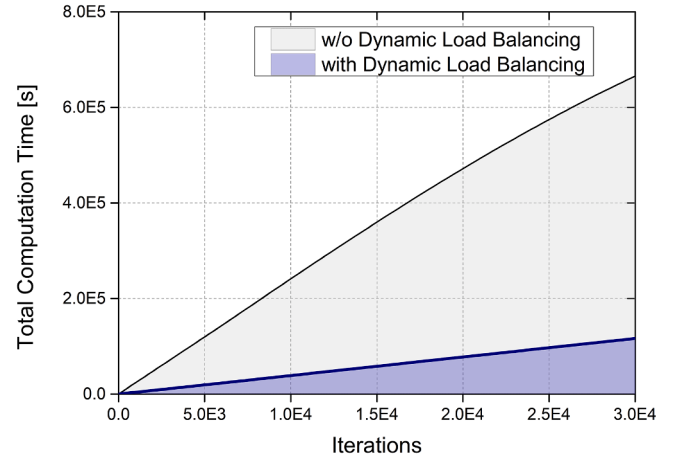


Fig. 37. Total computation time with and without the DLB algorithm.

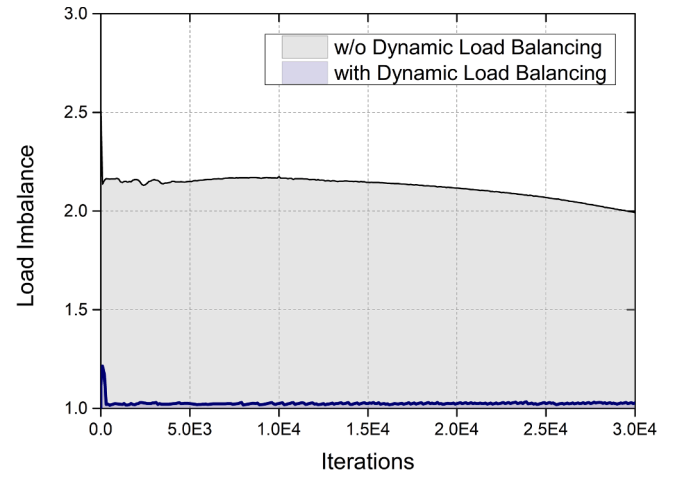


Fig. 38. Comparison of load imbalance with and without DLB.

resulting in 1,057,443,043 particles. The speed of sound and time step were selected as 90 m/s and 10^{-4} s to ensure numerical stability and CFL condition compliance. For parallelization, the domain was divided into 7 columns in the x-direction and 8 rows in the y-direction, utilizing a total of 56 GPUs.

Fig. 40 presents a rendered visualization of the one-billion-particle simulation, while Fig. 39 shows a top-view of the 2D domain partitioning, illustrating how GPU boundaries evolve dynamically over time. (See Video 1 for the full animation of the one-billion-particle simulation and Video 2 for the detailed visualization of the boundary adjustment process among GPUs.) With the DLB algorithm applied, the average load imbalance is 1.0337, which is close to the ideal value of 1.0, indicating an almost perfectly uniform workload distribution, as shown in Fig. 41. Even at this unprecedented scale, the proposed algorithm rapidly mitigated imbalance and maintained an even particle distribution, confirming its effectiveness and robustness for enabling practical, ultra-large-scale SPH simulations.

In practical terms, the one-billion-particle simulation was carried out for 100,000 time steps with a fixed time-step size of 10^{-4} s, corresponding to a physical simulation time of 10 s, and required a total computation time of 1.05×10^5 s (approximately 29.2 hours) on 56 GPUs.

5.4. Strong and weak scalability analysis

To evaluate the parallel scalability of the proposed DLB framework,

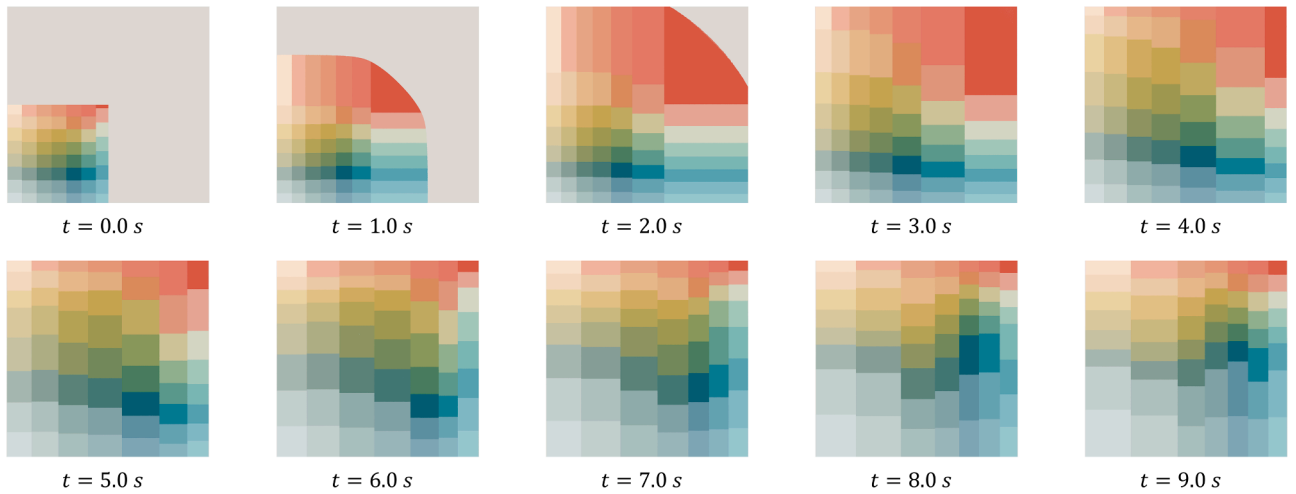


Fig. 39. Domain decomposition evolution in the one-billion-particle SPH simulation.

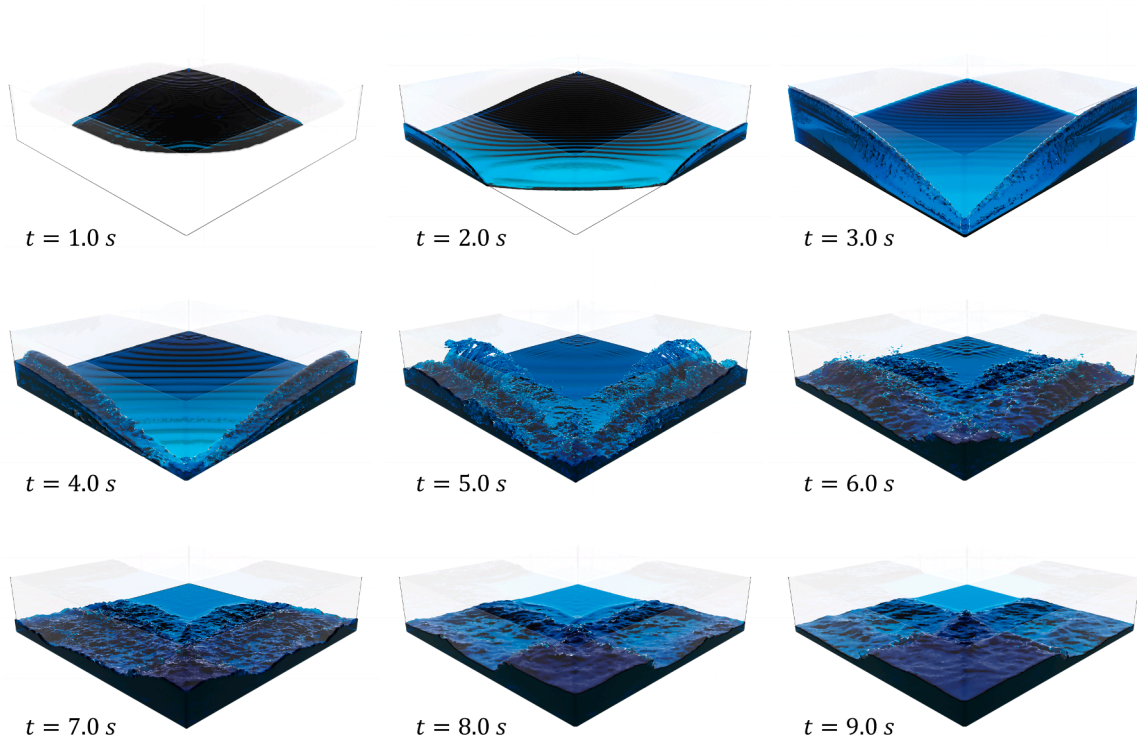


Fig. 40. Rendered visualization of the one-billion-particle SPH simulation.

both strong scalability and weak scalability analyses were conducted using 3D dam-break simulations. All simulations were conducted on the multi-GPU computing platform System A, which is described in detail in Section 5.1. In this study, strong scaling tests were conducted using a fixed total particle count of 20 million while varying the number of GPUs. All simulations employed identical physical models, particle resolutions, and time-step sizes. Only the number of GPUs was varied. For the weak scaling tests, the number of particles per GPU was fixed at 10 million and 20 million. The total number of particles increased linearly with the number of GPUs.

Fig. 42 presents the strong scalability performance of the proposed multi-GPU framework in terms of achieved speedup. As the number of GPUs increases, the total computation time decreases close to the ideal linear scaling trend, while the achieved speedup grows almost linearly, indicating that the additional computational resources are effectively

utilized.

Minor deviations from perfect linear scalability are primarily attributed to increased inter-GPU communication and synchronization overhead, as well as to the additional memory requirements associated with buffer particles introduced for halo region handling in the multi-GPU parallelization process. As the number of GPUs increases, the parallel efficiency, defined as the achieved speedup divided by the number of GPUs, gradually decreases, which is a behavior commonly observed in strong scaling environments. Nevertheless, the parallel efficiency remains approximately 96% for 6 GPUs, over 92% for 16 GPUs, and about 89% even when scaling up to 32 and 40 GPUs.

In particular, increasing the number of GPUs leads to a reduction in the number of particles per sub-domain, resulting in an increased fraction of particles involved in inter-GPU communication relative to the total particle count. Although this effect causes a gradual decrease in

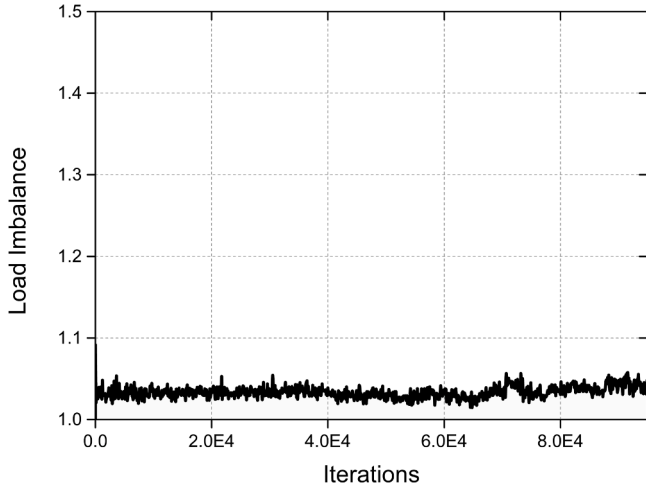


Fig. 41. Temporal evolution of load imbalance in the one-billion-particle simulation.

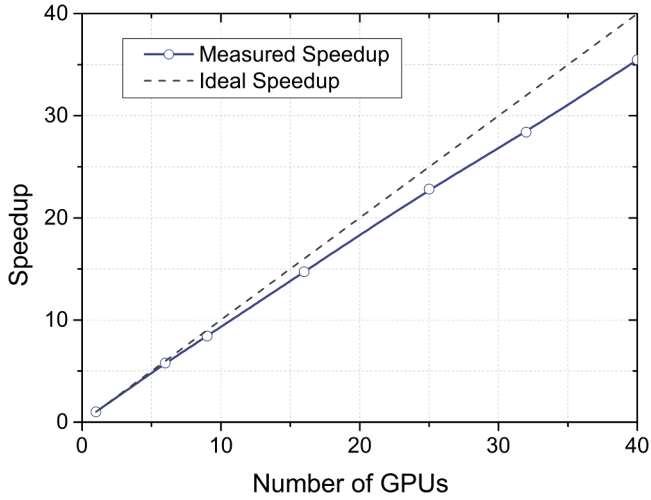


Fig. 42. Strong scaling performance of the proposed multi-GPU SPH framework.

parallel efficiency, the proposed DLB algorithm effectively alleviates load imbalance arising from particle redistribution, thereby preserving stable parallel efficiency even under such conditions. These results demonstrate that the proposed framework exhibits robust scalability against communication overhead and is well suited for large-scale multi-GPU SPH simulations.

Fig. 43 presents the weak scalability results of the proposed method for two cases in which the number of particles per GPU was fixed at 10 million and 20 million, respectively, while the total number of GPUs was increased to 1, 6, 16, 25, 32, 40, and 48. Accordingly, the total number of particles increased linearly with the GPU count.

The results show that the execution time remains nearly constant as the number of GPUs increases for both particle-loading conditions, demonstrating good weak scalability. In particular, slightly higher parallel efficiency is observed in the 20-million-particles-per-GPU case. This improvement is attributed to the reduced ratio of halo particles to total particles as the local subdomain size increases, which lowers communication overhead relative to computation. These results confirm that the proposed 2D domain decomposition and DLB strategy effectively mitigate load imbalance while controlling inter-GPU communication costs as the simulation scale grows.

The combined strong and weak scalability results demonstrate that

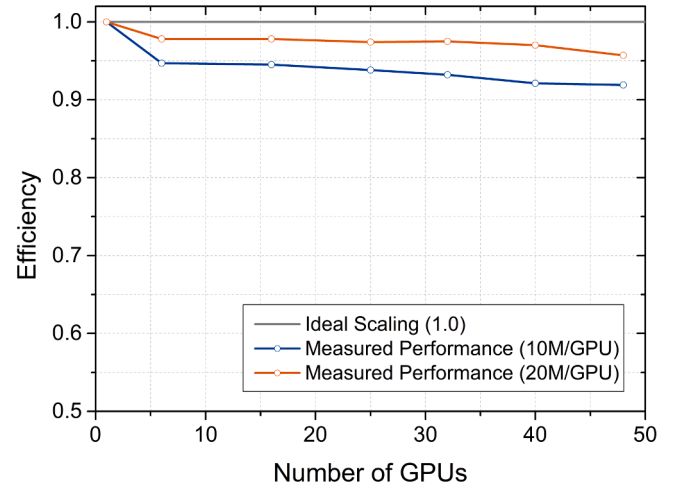


Fig. 43. Weak scaling performance of the proposed multi-GPU SPH framework.

the proposed framework achieves efficient parallel performance on multi-GPU systems under both fixed-size and growing problem configurations. By maintaining high parallel efficiency with a constant total particle count and stable computation time with increasing problem size, the proposed DLB algorithm proves to be well suited for large-scale SPH simulations involving highly non-uniform and time-dependent particle distributions.

To further analyze communication overhead, additional measurements were conducted focusing on the waiting time associated with MPI collective communication, particularly the MPI_Iscatterv routine used for particle data exchange during DLB. These measurements were performed on System A using 32 GPUs, with particle counts of 10 million and 20 million per GPU. The relative waiting-time imbalance was defined as the ratio of the maximum to the average waiting time across GPUs minus one, thereby quantifying the degree of load imbalance induced by communication synchronization.

Fig. 44 presents the measured waiting-time fraction and the relative waiting-time imbalance for the two particle-loading conditions. The results indicate that both the waiting-time fraction and the associated imbalance are consistently lower in the 20-million-particles-per-GPU case than in the 10-million-particles-per-GPU case. This behavior is expected, since the computational workload scales with the sub-domain volume, whereas inter-GPU communication is primarily governed by the interface area between neighboring sub-domains. As the number of

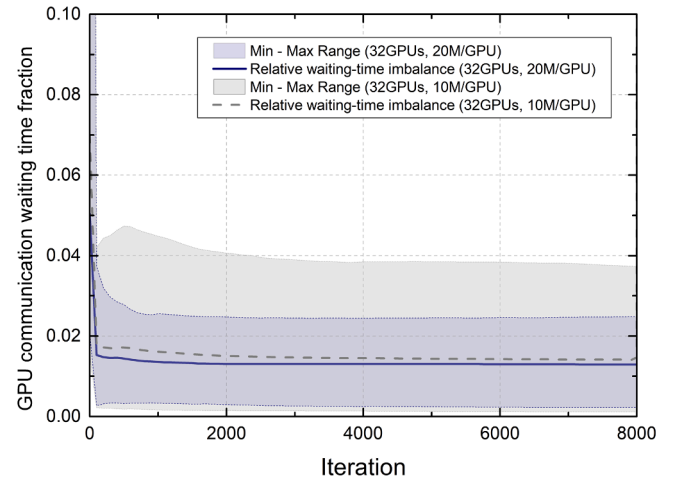


Fig. 44. Waiting-time fraction and relative waiting-time imbalance of MPI_Iscatterv communication using 32 GPUs for 10 million and 20 million particles per GPU.

particles per GPU increases, the relative contribution of communication overhead decreases, resulting in a smaller waiting-time fraction and reduced waiting-time imbalance. In addition, as observed in the figure, the absolute waiting-time fraction remains very small for both particle-loading conditions relative to the total execution time. This indicates that the MPI_Iscatterv-based collective communication is effectively overlapped with computation, and that the communication overhead is rarely translated into actual idle time.

Taken together, the strong and weak scalability results, along with the detailed communication analysis, demonstrate that the proposed 2D domain decomposition and DLB framework effectively mitigates load imbalance while controlling inter-GPU communication costs. These characteristics make the proposed framework well suited for large-scale multi-GPU SPH simulations involving highly non-uniform and time-dependent particle distributions.

To further assess the efficiency of the proposed parallel framework, a detailed breakdown of the computational cost was performed using weak scaling tests with 20 million particles per GPU, while varying the number of GPUs from 6 to 48. Fig. 45 presents the fraction of total execution time spent in mass and EOS evaluation, particle interaction computation, and inter-GPU communication.

The results show that mass and EOS evaluation account for approximately 17% of the total execution time, whereas particle interaction computation dominates the workload, contributing about 60%. This distribution reflects the inherently neighbor-based computational structure of SPH, in which most operations involve repeated loops over neighboring particles. Inter-GPU communication also occupies a comparable fraction of the raw execution timeline; however, this does not imply poor parallel efficiency.

As illustrated in Fig. 45, the majority of communication is successfully overlapped with computation. Consequently, the actual waiting (idle) time caused by communication synchronization is very small, as further demonstrated by the waiting-time analysis in Fig. 44. Even at large GPU counts, the waiting-time fraction remains negligible, indicating effective latency hiding and well-balanced communication patterns.

These results demonstrate that the proposed 2D DLB strategy does not merely redistribute computational workload but also enables efficient overlap of communication and computation. As a result, communication overhead does not translate into significant idle time, ensuring stable and efficient parallel performance at large scales.

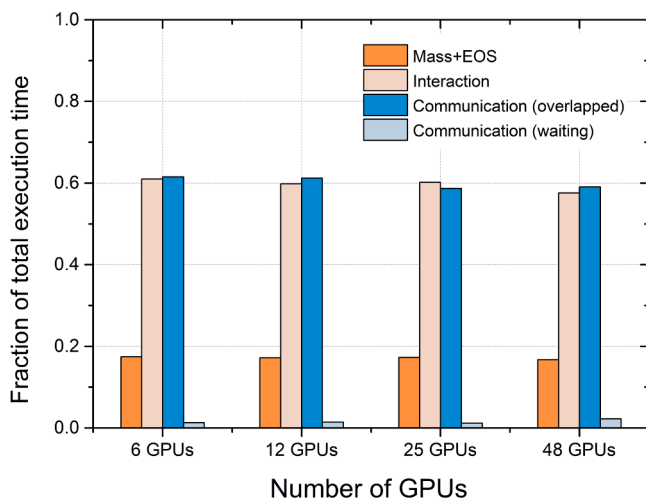


Fig. 45. Breakdown of computational cost in weak scaling tests with 20 million particles per GPU.

5.5. Application to large-scale multiphysics melt spreading in a core catcher

To demonstrate the applicability of the proposed DLB algorithm to realistic large-scale multiphysics problems, an engineering-scale simulation involving strongly coupled fluid flow, heat transfer, and phase-change effects was conducted. As a representative and challenging application, a molten corium spreading scenario in a reactor core catcher was selected.

In severe accident conditions, molten corium may be released from the reactor pressure vessel and relocate into a core catcher, a dedicated structure designed to retain, spread, and cool the melt. From a numerical perspective, this scenario represents an extreme multiphysics problem characterized by high temperatures, strong inertia, very high viscosity, complex geometry, and rapidly evolving free surfaces. The ability to reliably simulate such conditions is therefore an important benchmark for advanced multiphysics simulation frameworks.

The problem involves tightly coupled physical processes, including inertial-dominated free-fall motion, impact-driven free-surface deformation, viscosity-dominated spreading, and strong fluid-structure interaction with complex boundaries. These interacting mechanisms generate highly transient and spatially heterogeneous flow features, such as large interface deformation, localized high-shear regions, and asymmetric material redistribution. To achieve physically meaningful and numerically reliable predictions, high spatial resolution is essential, which in turn leads to extremely large particle counts and challenging computational load imbalance.

While core catcher geometries vary depending on reactor design and national implementation, a representative configuration was reconstructed based on available reference data. In the present setup, the lower head of the reactor pressure vessel is located approximately 7 m above the core catcher floor, such that the relocation process is dominated by free-fall motion following vessel breach. This vertical separation causes the molten corium to enter the core catcher with substantial downward momentum, resulting in intense impact loads and pronounced free-surface deformation immediately after relocation.

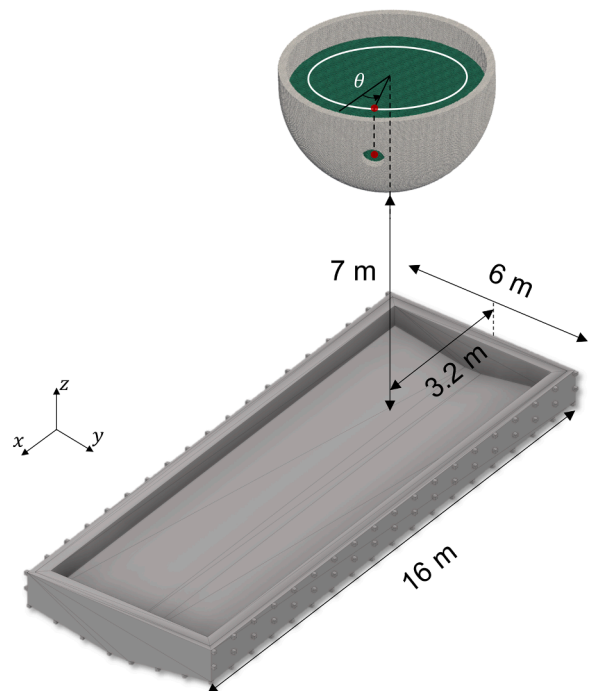


Fig. 46. Schematic configuration of the ex-vessel core catcher and reactor pressure vessel, including the rectangular catcher geometry and vessel breach orientations.

As illustrated in Fig. 46, the core catcher has a rectangular footprint with approximate dimensions of 6 m in width and 16 m in length, whereas the reactor pressure vessel is cylindrical. Due to this geometric mismatch, the corium enters the catcher in an inherently asymmetric manner. Furthermore, the impact location and initial spreading direction strongly depend on the angular position of the vessel breach. In this study, two representative breach orientations were considered, corresponding to 0° and 60° measured in the horizontal plane relative to the longitudinal axis of the core catcher, as schematically shown in Fig. 46. Conservative initial conditions were derived from prior system-level severe accident progression studies [51].

These impact-driven and direction-dependent flow characteristics result in highly non-uniform and time-dependent particle distributions throughout the simulation. Regions of high particle concentration and intense interaction continuously emerge and migrate as the melt spreads over the catcher surface. Consequently, the computational workload becomes persistently imbalanced in both space and time, rendering static domain decomposition strategies inefficient and making runtime DLB indispensable.

To account for phase-change effects of corium in a simplified yet robust manner, the effective viscosity of the melt was progressively increased during the simulation. However, the resulting extremely high viscosity imposes severe time-step restrictions when explicit integration schemes are used. To overcome this limitation, an implicit viscous solver was employed within the WCSPPH framework. The implicit formulation leads to a sparse linear system in which the particle velocities at the next time step are the unknowns. The system matrix is strictly diagonally dominant and symmetric positive definite, ensuring stable convergence of iterative solvers.

To avoid the excessive memory cost associated with storing large sparse matrices, a matrix-free conjugate gradient (CG) solver was adopted. Jacobi and block-Jacobi preconditioners were applied to accelerate convergence, with 3×3 block structures used to preserve local spatial coupling among neighboring particles. This strategy significantly

improves numerical robustness and computational efficiency in large-scale SPH simulations involving highly viscous fluids.

The simulation was performed using 15,727,212 particles with an initial particle spacing of 1.6 cm and executed in parallel on 24 GPUs of System A, a multi-GPU high-performance computing platform. Fig. 47 (a) and 47(b) present the simulation results corresponding to vessel breach angles of 0° and 60° , respectively. The results clearly illustrate the relocation process in which the molten corium, composed of oxide and metal layers, detaches from the reactor pressure vessel and subsequently falls into and spreads over the core catcher. Depending on the breach orientation, the initial impact location and spreading direction of the corium differ significantly, leading to distinct redistribution patterns within the core catcher.

This engineering-scale application demonstrates that the proposed approach is not limited to idealized benchmark problems but is directly applicable to large-scale SPH simulations involving complex geometries, extreme material properties, and strongly non-uniform particle redistribution.

6. Conclusion

In multi-GPU parallelized SPH simulations with fixed domain decomposition, a workload imbalance issue is inherently present. This is attributed to the movement of particles during simulation, causing variations in the spatial distribution of the workload. Accordingly, to address this intrinsic limitation, a DLB algorithm was developed in this study. MPI and CUDA were utilized to connect GPUs across multiple nodes in a network, enabling parallel computation.

To reduce communication overhead and improve computational efficiency, a 2D staggered domain decomposition strategy was introduced, together with an explicit overlap of communication and computation. Particle data required for inter-GPU communication were organized into a single contiguous array to minimize memory usage and facilitate efficient communication-computation overlap. All particle



Fig. 47. Engineering-scale SPH simulation of corium relocation into the core catcher under different vessel breach orientations.

exchanges were then performed using a single collective MPI_Isscatterv call. In addition, two complementary optimization strategies were incorporated: (i) a global, execution-time-based cumulative load metric for boundary adjustment, which maintains consistent workload distribution across all GPUs and mitigates long-range imbalance, and (ii) a dynamic cell resizing scheme for neighbor search, which adaptively adjusts cell sizes as particle distributions evolve, thereby reducing unnecessary neighbor search operations and lowering the per-iteration computational cost.

The effectiveness of the proposed framework was systematically validated through large-scale dam-break simulations. In the 79-million-particle case, the load imbalance metric was reduced from 1.575 to 1.026, and the computational performance was accelerated by up to $3.57\times$. Consistent improvements were observed in the 210-million-particle simulation, where the computation was accelerated by approximately $5.7\times$ while maintaining the load imbalance close to the ideal value of 1.0. Furthermore, compared with 1D domain decomposition, the proposed 2D approach reduced communication cost by a factor of 2.46 and improved overall computational efficiency by 1.54, clearly demonstrating the necessity and effectiveness of 2D decomposition for large-scale SPH simulations.

Beyond these benchmarks, the scalability and robustness of the proposed framework were further demonstrated through a one-billion-particle simulation executed on 56 GPUs. This result goes beyond a simple increase in problem size and shows that the proposed strategy can effectively suppress both computational and communication bottlenecks while sustaining high parallel efficiency under extreme workloads. The successful execution of this simulation confirms that SPH analyses at scales previously considered impractical can now be performed in a stable and efficient manner, while faithfully resolving high-resolution physical phenomena. In this regard, the one-billion-particle simulation represents an important demonstration for the practical applicability of SPH-based methods to ultra-large-scale problems.

Although the proposed framework demonstrates strong performance and scalability, several structural limitations should also be acknowledged. The proposed 2D staggered domain decomposition does not increase the number of neighboring GPUs per subdomain as the total number of GPUs grows, and therefore the communication topology itself does not become intrinsically more complex. However, when the number of GPUs increases to the order of several hundreds or more, the surface-to-volume ratio of each subdomain inevitably increases. As a result, the fraction of halo particles relative to interior particles gradually grows, leading to a relative increase in communication volume and synchronization overhead. Consequently, the effectiveness of computation-communication overlap is progressively reduced, which may impose structural constraints on further improvements in strong-scaling efficiency.

Nevertheless, as demonstrated by the weak-scaling analysis, the proposed framework remains highly effective for practical large-scale simulations. When maintaining approximately 20 million particles per GPU, a high parallel efficiency of about 96% was stably sustained up to 48 GPUs. This result indicates that, given sufficiently large problem sizes, the proposed approach can deliver high efficiency in realistic large-scale simulation environments targeting engineering applications. Therefore, while strong scaling may ultimately be limited by communication-dominated effects, the proposed framework provides a highly effective solution for large-scale SPH simulations involving hundreds of millions to billions of particles.

Finally, it should be noted that the proposed DLB strategy is heuristic in nature and does not aim to provide rigorous theoretical guarantees such as convergence proofs or optimality analysis. This design choice is motivated by the intrinsic characteristics of large-scale SPH simulations, in which particle distributions evolve continuously and nonlinearly due to free-surface motion, strong advection, and mixing. In such environments, accurately predicting future workload distributions or deriving provably optimal load balancing strategies in advance is impractical.

Consequently, heuristic-based approaches represent a practical and robust solution for maintaining scalability under time-varying workloads.

Future work will focus on extending the proposed framework to overcome these structural limitations by incorporating topology-aware communication strategies and predictive workload models, with the goal of achieving efficient scalability in ultra-large-scale parallel environments involving hundreds to thousands of GPUs.

CRediT authorship contribution statement

Yelyn Ahn: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Namjae Choi:** Conceptualization. **Hee Sang Yoo:** Visualization, Validation. **Eung Soo Kim:** Writing – review & editing, Supervision, Software, Resources, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Eung Soo Kim reports financial support was provided by National Research Foundation of Korea(NRF). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grants funded by the Korea government (MSIT) (Grant Nos. 2021M2D2A1A03046881 and RS-2024-00436693)..

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.advengsoft.2026.104123](https://doi.org/10.1016/j.advengsoft.2026.104123).

Data availability

No data was used for the research described in the article.

References

- [1] Gingold RA, Monaghan JJ. Smoothed particle hydrodynamics: theory and application to non-spherical stars. *Mon Not R Astron Soc* 1977;181:375–89. <https://doi.org/10.1093/mnras/181.3.375>.
- [2] Lucy LB. A numerical approach to the testing of the fission hypothesis. *Astron J* 1977;82:1013–24. <https://doi.org/10.1086/112164>.
- [3] Monaghan JJ, Kocharyan A. SPH simulation of multi-phase flow. *Comput Phys Commun* 1995;87:225–35. [https://doi.org/10.1016/0010-4655\(94\)00174-Z](https://doi.org/10.1016/0010-4655(94)00174-Z).
- [4] Violeau D, Rogers BD. Smoothed particle hydrodynamics (SPH) for free-surface flows: past, present and future. *J Hydraul Res* 2016;54:1–26. <https://doi.org/10.1080/00221686.2015.1119209>.
- [5] Monaghan JJ. Simulating free surface flows with SPH. *J Comput Phys* 1994;110:399–406. <https://doi.org/10.1006/jcph.1994.1034>.
- [6] Ye T, et al. Smoothed particle hydrodynamics (SPH) for complex fluid flows: recent developments in methodology and applications. *Phys Fluids* 2019;31:011301. <https://doi.org/10.1063/1.5068697>.
- [7] Vacondio R, et al. Grand challenges for Smoothed Particle Hydrodynamics numerical schemes. *Comput Part Mech* 2021;8:575–88. <https://doi.org/10.1007/s40571-020-00354-1>.
- [8] Harada T, Koshizuka S, Kawaguchi Y. Smoothed particle hydrodynamics on GPUs. *Comput Graphics Int. SBC Petropolis*. 2007;40:63–70.
- [9] Crespo AC, et al. GPUs, a new tool of acceleration in CFD: efficiency and reliability on smoothed particle hydrodynamics methods. *PLoS One* 2011;6:e20685. <https://doi.org/10.1371/journal.pone.0020685>.
- [10] Khrapov S, Khoperskov A. Smoothed-particle hydrodynamics models: implementation features on GPUs. Supercomputing. RuSCDays 2017. In: Voevodin V, Sobolev S, editors. Communications in computer and information

- science. Communications in computer and information science, 793; 2017. p. 266–77. https://doi.org/10.1007/978-3-319-71255-0_21.
- [11] Ihmsen M, et al. A parallel SPH implementation on multi-core CPUs. Computer Graphics Forum 2011;30:99–112. <https://doi.org/10.1111/j.1467-8659.2010.01832.x>.
 - [12] Jo YB, et al. SOPHIA: development of Lagrangian-based CFD code for nuclear thermal-hydraulics and safety applications. Ann Nucl Energy 2019;124:132–49. <https://doi.org/10.1016/j.anucene.2018.09.005>.
 - [13] Park SH, et al. Development of multi-GPU-based smoothed particle hydrodynamics code for nuclear thermal hydraulics and safety: potential and challenges. Front Energy Res 2020;8:86. <https://doi.org/10.3389/fenrg.2020.00086>.
 - [14] Crespo AJC, et al. DualSPHysics: open-source parallel CFD solver based on smoothed particle hydrodynamics (SPH). Comput Phys Commun 2015;187: 204–16. <https://doi.org/10.1016/j.cpc.2014.10.004>.
 - [15] Domínguez JM, Crespo AJC, Gómez-Gesteira M. Optimization strategies for CPU and GPU implementations of a smoothed particle hydrodynamics method. Comput Phys Commun 2013;184:617–27. <https://doi.org/10.1016/j.cpc.2012.10.015>.
 - [16] Guo C, Zhang H, Sun S, Qian Z, Liu M. A hybrid compressible-incompressible SPH model with adaptive iteration and GPU optimization for multiphase flows. J Comput Phys 2026;546:114513. <https://doi.org/10.1016/j.jcp.2025.114513>.
 - [17] Lu Y, Wu P, Liu M, Zhu C. A GPU-accelerated 3D ISPH-TLSPH framework for patient-specific simulations of cardiovascular fluid–structure interactions. Comput Methods Appl Mech Eng 2024;428:117110. <https://doi.org/10.1016/j.cma.2024.117110>.
 - [18] Zhang H, Zhang Z, He F, Liu M. Numerical investigation on the water entry of a 3D circular cylinder based on a GPU-accelerated SPH method. Eur J Mech B Fluids 2022;94:1–16. <https://doi.org/10.1016/j.euromechflu.2022.01.007>.
 - [19] H Huang C, et al. Coupled particle and mesh method in an Euler frame for unsteady flows around the pitching airfoil. Eng Anal Bound Elem 2022;138:159–76. <https://doi.org/10.1016/j.enganabound.2022.02.011>.
 - [20] Zhang Y, et al. Numerical simulation of casting filling process based on SPH–FEM coupling method. Symmetry 2025;17(4):494. <https://doi.org/10.3390/sym17040494>.
 - [21] Vacondio R, et al. Variable resolution for SPH: a dynamic particle coalescing and splitting scheme. Comput Methods Appl Mech Eng 2013;256:132–48. <https://doi.org/10.1016/j.cma.2012.12.014>.
 - [22] Kim DH, Ahn Y, Kim ES. Development of GPU-paralleled multi-resolution techniques for Lagrangian-based CFD code in nuclear thermal-hydraulics and safety. Nucl Eng Technol 2024;56(7):2498–515. <https://doi.org/10.1016/j.net.2024.02.008>.
 - [23] Amada T, et al. Particle-based fluid simulation on GPU. ACM Workshop on General-Purpose Computing on Graphics Processors. New York: ACM Press; 2004. p. 41.
 - [24] Héroult A, Bilotta G, Dalrymple RA. SPH on GPU with CUDA. J Hydraul Res 2010; 48(S1):74–9. <https://doi.org/10.1080/00221686.2010.9641247>.
 - [25] Valdez-Balderas D, et al. Towards accelerating smoothed particle hydrodynamics simulations for free-surface flows on multi-GPU clusters. J Parallel Distrib Comput 2013;73:1483–93. <https://doi.org/10.1016/j.jpdc.2012.07.010>.
 - [26] Verma K, et al. A multi-GPU PCISPH implementation with efficient memory transfers. In: 2018 IEEE High Performance Extreme Computing Conference (HPEC). IEEE; 2018. <https://doi.org/10.1109/HPEC.2018.8547542>.
 - [27] Oger G, et al. On distributed memory MPI-based parallelization of SPH codes in massive HPC context. Comput Phys Commun 2016;200:1–14. <https://doi.org/10.1016/j.cpc.2015.08.021>.
 - [28] Cherfilis JM, Pinon G, Rivoalen EJOSEPHINE. A parallel SPH code for free-surface flows. Comput Phys Commun 2012;183:1468–80. <https://doi.org/10.1016/j.cpc.2012.02.007>.
 - [29] Ferrari A, et al. A new 3D parallel SPH scheme for free surface flows. Comput Fluids 2009;38:1203–17. <https://doi.org/10.1016/j.compfluid.2008.11.012>.
 - [30] Domínguez JM, et al. New multi-GPU implementation for smoothed particle hydrodynamics on heterogeneous clusters. Comput Phys Commun 2013;184: 1848–60. <https://doi.org/10.1016/j.cpc.2013.03.008>.
 - [31] Verma K, Szewc K, Wille R. Advanced load balancing for SPH simulations on multi-GPU architectures. In: 2017 IEEE High Performance Extreme Computing Conference (HPEC). IEEE; 2017. <https://doi.org/10.1109/HPEC.2017.8091093>.
 - [32] Tsuzuki S, Aoki T. Effective dynamic load balance using space-filling curves for large-scale SPH simulations on GPU-rich supercomputers. 2016 7th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA). IEEE; 2016. <https://doi.org/10.1109/ScalA.2016.005>.
 - [33] Guo X, et al. New massively parallel scheme for incompressible smoothed particle hydrodynamics (ISPH) for highly nonlinear and distorted flow. Comput Phys Commun 2018;233:16–28. <https://doi.org/10.1016/j.cpc.2018.06.006>.
 - [34] Furuichi M, Nishiura D. Iterative load-balancing method with multigrid level relaxation for particle simulation with short-range interactions. Comput Phys Commun 2017;219:135–48. <https://doi.org/10.1016/j.cpc.2017.05.015>.
 - [35] Sasidharan A, Dennis JM, Snir M. A general space-filling curve algorithm for partitioning 2D meshes. In: 2015 IEEE 17th International Conference on High Performance Computing and Communications. IEEE; 2015. <https://doi.org/10.1109/HPCC-CSS-ICSS.2015.192>.
 - [36] Borrell R, et al. SFC-based multi-partitioning for accurate load balancing of CFD simulations. In: Tenth International Conference on Computational Fluid Dynamics (ICCFD10); 2018.
 - [37] Egorova MS, et al. Parallel SPH modeling using dynamic domain decomposition and load balancing displacement of Voronoi subdomains. Comput Phys Commun 2019;234:112–25. <https://doi.org/10.1016/j.cpc.2018.07.019>.
 - [38] Muratov RV, Ryabov PN, Dyachkov SA. Dynamic domain decomposition method based on weighted Voronoi diagrams. Comput Phys Commun 2023;290:108790. <https://doi.org/10.1016/j.cpc.2023.108790>.
 - [39] Monaghan JJ, Kos A. Solitary waves on a Cretan beach. J Waterw Port Coast Ocean Eng 1999;125:145–55. [https://doi.org/10.1061/\(ASCE\)0733-950X\(1999\)125:3\(145\)](https://doi.org/10.1061/(ASCE)0733-950X(1999)125:3(145)).
 - [40] Dehnen W, Aly H. Improving convergence in smoothed particle hydrodynamics simulations without pairing instability. Mon Not R Astron Soc 2012;425:1068–82. <https://doi.org/10.1111/j.1365-2966.2012.21439.x>.
 - [41] Domínguez JM, et al. Neighbour lists in smoothed particle hydrodynamics. Int J Numer Methods Fluids 2011;67:2026–44. <https://doi.org/10.1002/fld.2481>.
 - [42] Green S. Particle simulation using CUDA. NVIDIA Whitepaper 2010;6:121–8.
 - [43] Yoo HS, Kim ES. Coupled EISPH–OSPD framework for modeling hydroelastic response and brittle fracture. Comput Mech 2025;1–35. <https://doi.org/10.1007/s00466-025-02697-z>.
 - [44] Domínguez JM, et al. DualSPHysics: from fluid dynamics to multiphysics problems. Comput Part Mech 2022;9(5):867–95. <https://doi.org/10.1007/s40571-021-00404-2>.
 - [45] Kleefsman KMT, et al. A volume-of-fluid based simulation method for wave impact problems. J Comput Phys 2005;206(1):363–93. <https://doi.org/10.1016/j.jcp.2004.12.007>.
 - [46] Open MPI Documentation. MPI_Iscatterv (Version 4.1). https://www.open-mpi.org/doc/v4.1/man3/MPI_Iscatterv.3.php; accessed October 2025.
 - [47] Zhu G, et al. A novel MPI-based parallel smoothed particle hydrodynamics framework with dynamic load balancing for free surface flow. Comput Phys Commun 2023;284:108608. <https://doi.org/10.1016/j.cpc.2022.108608>.
 - [48] Ahn Y, et al. Dynamic load balancing of multi-GPU parallelization for VULCANO VE-U7 corium spreading analysis using SOPHIA. Nucl Technol 2025;211:1316–36. <https://doi.org/10.1080/00295450.2024.2397617>.
 - [49] Open MPI Documentation. MPI_Alltoall (Version 4.0). https://www.open-mpi.org/doc/v4.0/man3/MPI_Alltoall.3.php; accessed October 2025.
 - [50] Open MPI Documentation. MPI_Gather (Version 3.1). https://www.open-mpi.org/doc/v3.1/man3/MPI_Gather.3.php; accessed October 2025.
 - [51] Ryu CK, Ye IS, Hong JH, Kim JE. Evaluation of core melt spreading. Daejeon (KR): Korea Atomic Energy Research Institute; 2011. KAERI/CM–1417/2010.